

**FACULTE DES SCIENCES
DEPARTEMENT DE MATHEMATIQUE ET INFORMATIQUE**

Algorithmique

L2 LMD

Génie, E-commerce, IAG & Réseau Informatique

Phd, MAYALA LEMBA Francis

Professeur Associé

Système d'information ,Théorie des nombres & Sécurité Informatique

Département de Mathématique et Informatique

Université Pédagogique Nationale

MPEMBA NGOMA Luz

Chef de Travaux

Concepteur des systèmes d'information , IT-Manager & Data Analyst

LIKOTELO BINENE Camile

Assistant 2

DBA , Développeur & Data Analyst

L'objectif de ce support est d'initier le lecteur à la résolution des problèmes par la programmation, commençant par l'analyse du problème, la recherche de la solution ou la méthode pour résoudre ce problème, l'écriture de cette solution sous forme d'un algorithme, et enfin la traduction de l'algorithme en programme exécutable par la machine en utilisant un langage de programmation tel que le langage C, Python, Java....

Il convient de noter que ce support de cours est un manuel d'accompagnement de l'étudiant et sa lecture ne constitue en aucun cas une dispense de la présence attentive aux séances de cours et de travaux dirigés ou pratiques.

Première Partie :Cours

Chapitre Premier : Introduction aux algorithmes

1. Contexte

Le terme **Informatique** est un néologisme proposé en 1962 par *Philippe Dreyfu¹s* pour caractériser le traitement automatique de l'information, c'est une contraction de l'expression « information automatique ». Ce terme a été accepté par l'Académie française en avril 1966, et l'informatique devint alors officiellement la science du traitement automatique de l'information, où l'information est considérée comme le support des connaissances humaines et des communications dans les domaines techniques, économiques et sociaux [3].

2. Notions élémentaires

▪ Informatique

L'informatique est la science du traitement automatique de l'information. Elle traite de deux aspects complémentaires :

- les programmes ou logiciels (software) qui décrivent un traitement à réaliser, -
- les machines ou le matériel (hardware) qui exécute ce traitement.

▪ Hardware

C'est l'ensemble des éléments physiques (microprocesseur, mémoire, disques durs, ...) utilisés pour traiter les informations.

▪ Software

C'est un programme (ou ensemble de programmes) décrivant un traitement d'informations à réaliser par un matériel informatique.

▪ Algorithme

La notion d'algorithme est à la base de toute la programmation informatique [8]. La définition la plus simple que l'on peut associer à cette notion est qu'un algorithme est une suite ordonnée d'instructions qui indique la démarche à suivre pour résoudre un problème ou effectuer une tâche. Le mot algorithme vient du nom latinisé du mathématicien perse *AlKhawarizmi*, surnommé « le père de l'algèbre » [11].

Exemple : Appel téléphonique

- a. Ouvrir son téléphone,

¹ Informaticien Français

- b. Chercher/Composer le numéro du destinataire,
- c. Appuyer sur le bouton « Appel ».

Ce mode d'emploi précise comment faire un appel téléphonique. Il est composé d'une suite ordonnée d'instructions (ouvrir, chercher/composez, appuyer) qui manipulent des données (téléphone, numéro, bouton) pour réaliser la tâche d'appel.

3. L'algorithmique

3.1. Définition

L'algorithmique est la science des algorithmes. Elle s'intéresse à l'art de construire des algorithmes ainsi qu'à déterminer leur validité, leur robustesse, leur réutilisabilité, leur complexité ou leur efficacité [3]. L'algorithmique permet ainsi de passer d'un problème à résoudre à un algorithme qui décrit la démarche de résolution du problème. Par conséquent, la programmation consiste à traduire un algorithme dans un langage « compréhensible » par l'ordinateur afin qu'il puisse être exécuté automatiquement.

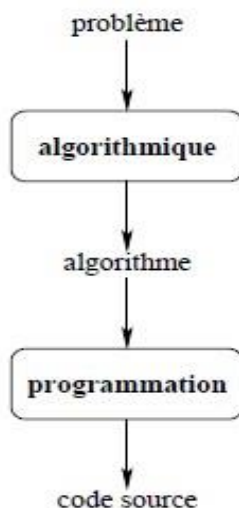


Figure 1. Etapes de développement.

La figure 1 ci-dessus illustre les deux phases nécessaires pour obtenir un code source :

- Phase d'algorithmique qui implique la recherche et l'écriture d'un algorithme ;
- Phase de programmation qui consiste à traduire l'algorithme obtenu en un programme à l'aide d'un langage de programmation (C, Java, Python,...).

Dans la première phase, on doit définir les données qu'on dispose et les objectifs qu'on souhaite atteindre, ainsi que prévoir des réponses à tous les cas possibles.

Exemple : résolution d'une équation de second degré $ax^2+bx+c=0$

→ Les données sont a, b et c

→ Les sorties sont x_1 et x_2

→ Les cas : $a=0$ et $b \neq 0$, $a = 0$ et $b = 0$, $a \neq 0$,

3.2. Principe général

Le traitement automatique de l'information consiste à exécuter des instructions (opérations élémentaires et complexes) sur des données d'entrée afin de générer d'autres informations appelées résultats ou données de sortie.

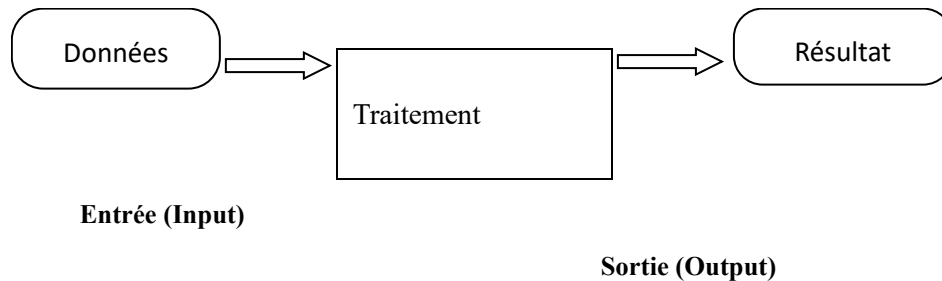


Figure 2. Principe du traitement automatisé.

Exemple : Calcul de la moyenne d'un étudiant

Supposons qu'on doit calculer la moyenne d'un étudiant pour un ensemble de matières.

Donc, on doit :

- i. Définir le nombre des matières concernées ainsi que les notes et les coefficients ;
- ii. Réaliser les opérations suivantes :
 - Multiplier chaque note d'une matière par son coefficient,
 - Calculer la somme des résultats des multiplications,
 - Diviser la somme obtenue par le total des coefficients,
- iii. Afficher la moyenne de l'étudiant (résultat final).

Remarque :

Lorsqu'on écrit un algorithme, les questions suivantes doivent être considérées :

- ✓ Quel est le résultat attendu ?
- ✓ Quelles sont les données nécessaires (informations requises) ?
- ✓ Comment faire (le traitement à réaliser) ?

4. Caractéristiques des algorithmes

4.1. Structure générale

Un algorithme se compose généralement de deux parties :

Partie déclarative : appelée aussi entête de l'algorithme, elle contient généralement les déclarations (des constantes, des variables, etc.).

Partie corps de l'algorithme : constituée d'une ou plusieurs séquences d'instructions faisant appel à des opérations de base à exécuter par l'ordinateur.

Syntaxe :

Algorithme nom_de_l'algorithme ;	}	Partie déclarative (Entête)
< Liste de variables/constantes > ;		
Début	}	Partie corps de l'algorithme Fin
< Séquence d'instructions > ;		

4.2. Les variables et les constantes

L'élément unitaire de stockage de l'information est appelé **bit**. Un bit ne peut avoir que deux états distincts : 0 ou 1 (vrai ou faux dans la logique). Dans la mémoire de l'ordinateur, les données sont manipulées par groupes de 8 bits (octets), ou plus (mots de 16, 32, 64 bits,...).

Une case mémoire est donc appelée **mot** et pour que l'unité centrale puisse stocker une information et la retrouver dans la mémoire, chaque mot est repéré par une **adresse** [4].

Dans la programmation, les adresses mémoire sont représentées par des noms. Le programmeur ne connaît pas donc l'adresse d'une case mais plutôt son nom. Il y a donc deux façons de voir la mémoire centrale de l'ordinateur : côté programmeur et côté ordinateur tel qu'il est illustré, à titre d'exemple, dans le schéma suivant (figure 3).

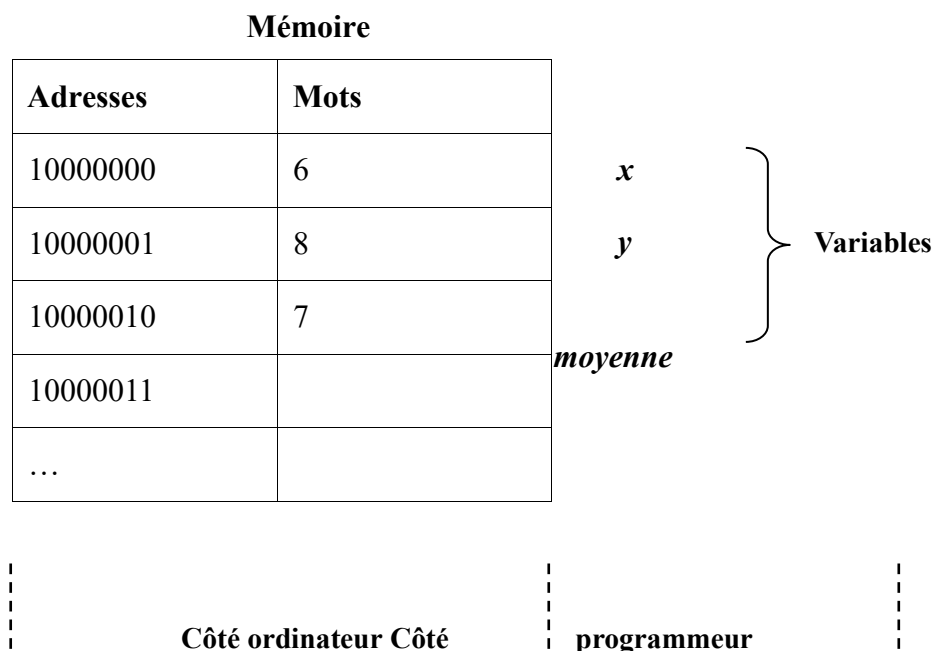


Figure 3. Organisation de la mémoire.

4.2.1. Les variables

Une variable est une case mémoire destinée à contenir des valeurs de type défini au préalable

(nombres, caractères, chaînes de caractères,...). Elle possède un **nom**, un **type**, et un **contenu** qui peut être modifié au cours de l'exécution de l'algorithme.

Le mot clé est: **Var**.

4.2.2. Les constantes

La définition d'une constante est la même que celle d'une variable à la différence que sa valeur reste inchangée tout au long du déroulement (exécution) de l'algorithme.

Le mot clé est: **Const**.

Les variables et les constantes sont déclarées selon la syntaxe suivante :

Syntaxe :

Var nom_variable : type ;

Const nom_constante = valeur ;

Remarque :

Dans la partie déclarative, les variables et les constantes sont caractérisées essentiellement par :

- **Un identificateur** : est un nom attribué à la variable ou à la constante, qui peut être composé de lettres et de chiffres mais sans espaces.
- **Un type** : qui définit la nature et la taille de la variable.

Exemple :

Var x, y : entier;

Const alpha = 0,5 ;

Dans cet exemple, nous avons déclaré :

- Deux variables (x et y) de type entier, ce type est décrit dans la sous-section suivante.
- Une constante (alpha) égale à la valeur 0,5 à titre d'exemple.

4.3. Les types de base

Le type d'une variable définit l'ensemble des valeurs que peut prendre la variable, ainsi que l'ensemble des opérations que l'on peut appliquer sur cette variable. Il existe des types simples prédéfinis tels que : entier, réel, caractère et booléen.

4.3.1. Type entier

C'est un type numérique représentant l'ensemble des entiers relatifs, tels que: -9, 0, 31,

Les opérations permises sur ce type sont : +, -, *, div (division entière) et mod (modulo ou reste de la division entière). Le mot clé est : **entier**.

Exemple : Var x : entier ;

4.3.2. Type réel

C'est un type numérique aussi représentant l'ensemble des nombres réels, tels que : 0.25,

-

1.33, 2.5×10^3 , Les opérations permises sur ce type sont : +, -, * et /.

Le mot clé est : **réel**.

Exemple : Var y : réel ;

4.3.3. Type caractère

Ce type représente tous les caractères alphanumériques tels que : 'a', 'A', '3', '%', ' ', ...

Les opérations supportées par ce type sont : =, ≠, <, <=, >, >=.

Le mot clé est : **caractère**.

Exemple : Var a : caractère ;

4.3.4. Type booléen

Ce type est utilisé dans la logique pour représenter les deux valeurs : vrai et faux.

Les opérations prises en charge sont : NON, ET, OU.

Le mot clé est : **booléen**.

Exemple : Var b : booléen ;

4.3.5. Chaîne de caractères

Ce type représente les mots et les phrases tels que "Algorithmique", "Cours", etc. Le mot clé utilisé est : **chaîne**

Exemple : Var c : chaîne ;

Globalement, la partie déclarative d'un algorithme peut être représentée comme suit.

Exemple :

```
Var  x, y : entier ;  
      z, w : réel ;  
      lettre : caractère ;  
      nom : chaîne ;  
      Etat : booléen ;  
Const n = 100 ;  
      arobase = '@' ;  
      mot = "bonjour"  
      ;
```

5. Conclusion

Ce chapitre constitue une initiation aux notions basiques de l'écriture des algorithmes. La syntaxe d'un algorithme, la notion de variable et de constante, ainsi que leurs types sont définis et expliqués à travers des exemples simples. Ceci constitue pour le lecteur un prérequis de base qui lui permettra de comprendre la notion d'instructions algorithmiques dans les prochains chapitres.

Chapitre Deuxième : Les instructions simples

1. Introduction

Un algorithme, par définition, est un ensemble d'instructions qui peuvent être simples ou complexes. Dans ce chapitre, on s'intéressera aux instructions simples notamment : les instructions d'affectation, de lecture et d'écriture.

2. L'instruction d'affectation

Cette instruction est élémentaire en algorithmique, elle permet d'assigner une valeur à une variable selon la syntaxe suivante :

variable $\leftarrow$ expression ;

Une instruction d'affectation est exécutée comme suit :

- évaluation de l'expression située à droite de l'instruction, et
- affectation du résultat à la variable située à gauche de l'instruction.

L'expression peut être :

- une constante ($c \leftarrow 10$)
- une variable ($v \leftarrow x$)
- une expression arithmétique ($e \leftarrow x + y$)
- une expression logique ($d \leftarrow a \text{ ou } b$)

Remarque :

- Une constante ne figure jamais à gauche d'une instruction d'affectation. Exemple d'instruction fautive : **Const** $z = 1$;
 $z \leftarrow 2$; « *Faux* »
- Après une affectation, l'ancien contenu d'une variable est substitué (écrasé) par le nouveau contenu. Exemple : **Var** $a : \text{entier}$; $a \leftarrow 1$; $a \leftarrow 2$;
Après la deuxième affectation, la valeur de a est devenue 2 (la valeur 1 est écrasée).
- Une instruction d'affectation doit se faire entre deux types compatibles.
Exemple : **Var** $x, y : \text{entier}$; $z : \text{réel}$; $a, b : \text{caractère}$;

Instructions correctes	Instructions incorrectes
------------------------	--------------------------

$x \leftarrow y ;$ $y \leftarrow x ;$ $z \leftarrow x ;$ $a \leftarrow b ;$ $b \leftarrow a ;$	$x \leftarrow z ;$ $x \leftarrow a ;$ $b \leftarrow y ; a$ $\leftarrow z ;$
--	--

Les expressions arithmétiques ou logiques sont composées d'au moins deux termes reliés par un ou plusieurs opérateurs dont on peut distinguer :

a) Les opérateurs arithmétiques (par ordre de priorité) :

$\wedge$ ou $**$: Puissance

$*$, $/$, mod : Multiplication, Division et Modulo

$+$, $-$: Addition et Soustraction

b) Les opérateurs logiques ou booléens :

NON : Non logique (négation)

ET : Et logique (conjonction)

OU : Ou logique (disjonction)

NON ET : négation de conjonction

NON OU : négation de disjonction

c) Les opérateurs de comparaison ou relationnels :

$>$, $\geq$: supérieur et supérieur ou égal

$<$, $\leq$: inférieur et inférieur ou égal

$=$, $\neq$ (ou $<>$) : égal et différent

Remarque :

Les expressions logiques peuvent être composées des opérateurs logiques et/ou relationnels. Par exemple, $(A < 20) \text{ ET } (B \geq 10)$ est Vrai si A est inférieur à 20 et B est égal ou supérieur à 10, et faux sinon.

3. L'instruction de lecture

Cette instruction est très primordiale dans un algorithme. Elle permet de lire des valeurs en entrée (*input*) et les affecter aux variables stockées dans la mémoire. Les valeurs affectées sont souvent des données introduites à partir d'un périphérique d'entrée tel que le clavier.

Syntaxe :

Lire (var1, var2,...) ;

Exemple :

Lire(x) : lit et stocke une valeur donnée dans la case mémoire associée à x.

Lire(x, y) : lit et stocke deux valeurs, la première dans x et la deuxième dans y.

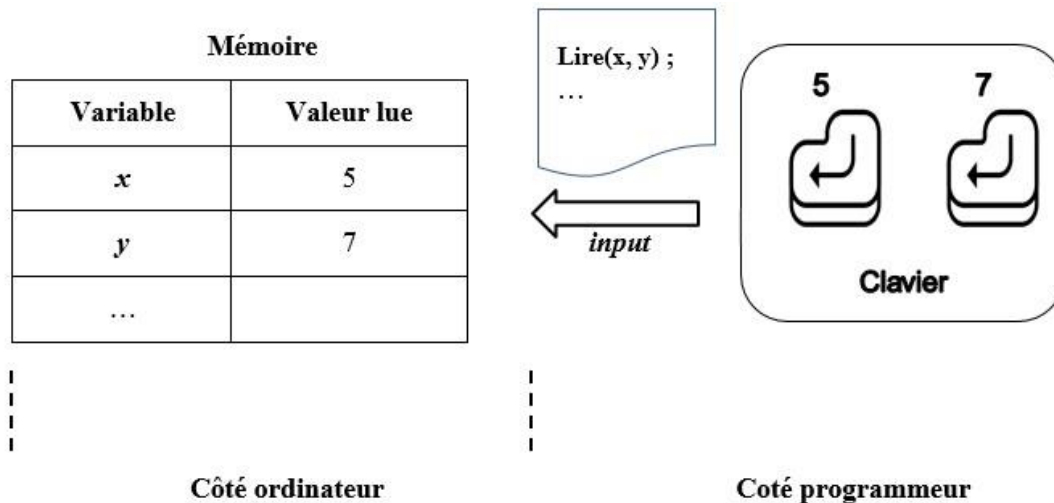
Illustration :

Figure 4. Opération de lecture.

4. L'instruction d'écriture

Cette instruction est aussi d'une grande importance dans les algorithmes. Elle permet d'écrire en sortie (*output*) les données résultant d'un traitement effectué par l'algorithme (valeur, texte, ...) en les affichant par exemple sur un périphérique de sortie tel que l'écran.

Syntaxe :

Ecrire (var1, var2, expr1, expr2, ...);

Remarque :

Dans le cas d'écriture d'une expression, c'est le résultat d'évaluation de cette expression qui est affiché et non pas l'expression elle-même. Par exemple :

Soient deux variables x et y tel que x=5 et y=7, l'instruction :

Ecrire (x+y) ;

Affiche en sortie le résultat d'addition de x et y (soit 12).

Illustration :

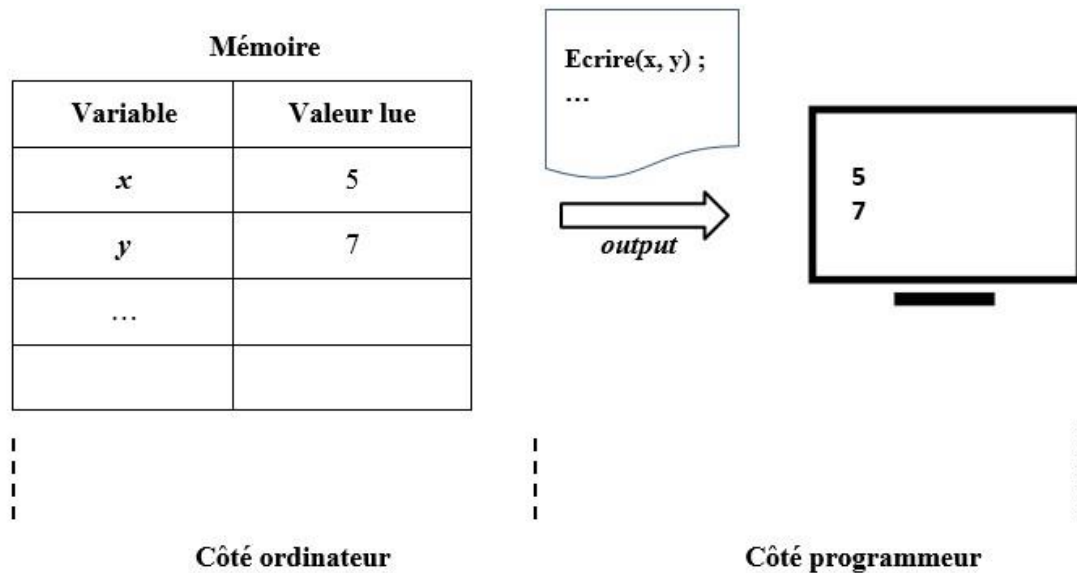


Figure 5. Opération d'écriture.

Exemple d'algorithme contenant les trois instructions précédentes :

Algorithme Moyenne_deux_réels

Var x, y, z : réel ;

Début

Ecrire ("Donner la première valeur :") ;

Lire (x) ;

Ecrire ("Donner la deuxième valeur :") ;

Lire (y) ; $z \leftarrow (x + y)/2$;

Ecrire ("La moyenne est : ", z) ;

// On peut remplacer les deux dernières instructions par une seule :

Ecrire ("La moyenne est : ", $(x + y)/2$) ; *// dans ce cas on a pas besoin de z* **Fin**

Dans cet algorithme, si l'utilisateur introduit 10 pour x et 20 pour y alors l'affichage sera :

La moyenne est : 15

5. Conclusion

Dans ce chapitre, nous avons présenté les instructions algorithmiques fondamentales à savoir l'affectation, la lecture et l'écriture. Ces trois simples instructions sont incontournables dans l'écriture d'un algorithme et constituent l'un des moyens les plus simples qui permettent au programmeur d'interagir avec son ordinateur à travers des actions d'entrées/sorties.

Chapitre troisième : Les instructions conditionnelles

(les alternatives)

1. Introduction

Les algorithmes comportent généralement deux types d'instructions :

- Les instructions simples : qui permettent la manipulation des variables telles que l'affectation, la lecture et l'écriture.
- Les instructions de contrôle : qui précisent l'enchaînement chronologique des instructions simples. C'est en particulier le cas des instructions conditionnelles ou les tests.

2. Structure d'un test

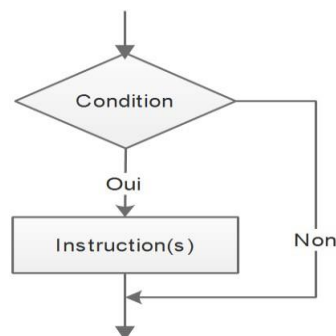
Il existe deux formes de test : forme simple (ou réduite) et forme complète.

2.1. Forme simple

Dans cette forme, une action qui correspond à une ou plusieurs instructions, est exécuté si une condition est vérifiée. Sinon l'algorithme passe directement au bloc d'instruction qui suit immédiatement le bloc conditionnel.

Syntaxe :

```
Si (condition) Alors instruction(s) ; //  
    action  
Finsi
```



Remarque :

La condition évaluée après l'instruction « Si » est une variable ou une expression booléenne qui, à un moment donné, est Vraie ou Fausse. par exemple : $x=y$; $x \leq y$; ...

Exemple :

```
x ← 5 ; y ← 9 ;  
Si (x = y) Alors Ecrire ("x est égale à y") ;
```

Dans cet exemple, le message « x est égale à y » ne sera pas affiché puisque la condition ($x = y$) n'est pas vérifiée.

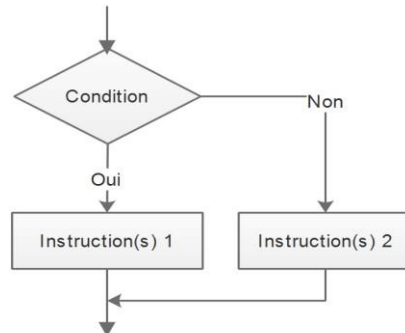
2.2. Forme complète

Cette forme permet de choisir entre deux actions selon qu'une condition est vérifiée ou non.

Syntaxe :

```

Si (condition) Alors
    instruction(s) 1 ; // action1
Sinon instruction(s) 2 ; // action2
Finsi
  
```



Remarque :

Certains problèmes exigent parfois de formuler des conditions qui ne peuvent pas être exprimées sous la forme d'une simple comparaison. Par exemple, la condition $x \in [0, 1[$ s'exprime par la combinaison de deux conditions $x \geq 0$ et $x < 1$ qui doivent être vérifiées en même temps.

Pour combiner ces deux conditions, on utilise les opérateurs logiques. Ainsi, la condition $x \in$

$[0, 1[$ pourra s'écrire sous la forme : $(x \geq 0) \text{ ET } (x < 1)$. Cette dernière est appelée une condition composée ou complexe.

Exemple (sur la forme complète d'un test) :

```

x ← 5 ; y ← 9 ;
Si (x = y) Alors Ecrire ("x est égale à y ") ;
Sinon Ecrire ("x est différente de y ") ;
  
```

Avec cette forme, on peut traiter les deux cas possibles. Si la condition $(x=y)$ est vérifiée, le premier message est affiché, si elle n'est pas vérifiée, le deuxième message est affiché.

3. Tests imbriqués

La forme « **Si ... Alors...Sinon** » permet deux choix correspondants à deux traitements différents. Dans d'autres situations, on pourra avoir plus de deux cas ce qui rend cette alternative insuffisante pour traiter tous les cas possibles (voir exemple ci-dessous). La forme complète permet de choisir entre plusieurs actions en imbriquant des formes simples selon la syntaxe ci-dessous.

Syntaxe :

```

Si (condition1) Alors instruction(s) 1 ;
  
```



```

Sinon Si (condition2) Alors instruction(s) 2 ;
      Sinon Si (condition3) Alors instruction(s) 3 ;
      ...
      Sinon instruction(s) N ;
      FinSi
FinSi
FinSi

```

Exemple : Etat de l'eau [10]

Dans les conditions normales de température et de pression, l'eau est sous forme de glace si la température est inférieure ou égale à 0° C, sous forme de liquide si la température est comprise entre 0° C et 100° C et sous forme de vapeur au-delà de 100° C. Ecrivons l'algorithme qui permet de vérifier l'état de l'eau selon sa température. La solution pourrait être comme suit :

```

Algorithme Etat_Eau ;
Var t : réel ;
Début
  Ecrire ("Donner la température de l'eau :") ;
  Lire (t) ;
  Si (t <= 0) Alors Ecrire ("Etat solide") ;
  FinSi
  Si (t > 0 ET t < 100) Alors Ecrire ("Etat liquide") ;
  Finsi
  Si (t >= 100) Alors Ecrire ("Etat gazeux") ;
  Finsi
Fin

```

Cet algorithme est correct mais il évalue les trois conditions qui portent sur la même variable et qui sont exclusives. En effet, si $(t \leq 0)$, alors on ne peut pas avoir $(t \geq 0 \text{ et } t < 100)$ ni $(t > 100)$. Il est donc inutile d'évaluer les deux dernières conditions si la première est vérifiée, ou d'évaluer la dernière condition si la deuxième est vérifiée. Pour éviter ce cas de figure, il sera préférable d'utiliser des tests imbriqués comme suit :

```

...
Début
  Ecrire ("Donner la température de l'eau:") ;
  Lire (t) ;
  Si (t <= 0) Alors Ecrire ("Etat solide") ;
  Sinon Si (t < 100) Alors Ecrire ("Etat liquide") ;
      Sinon Ecrire ("Etat gazeux") ;
  Finsi
Finsi

```

Fin

L'organigramme correspondant est illustré dans la figure 6.

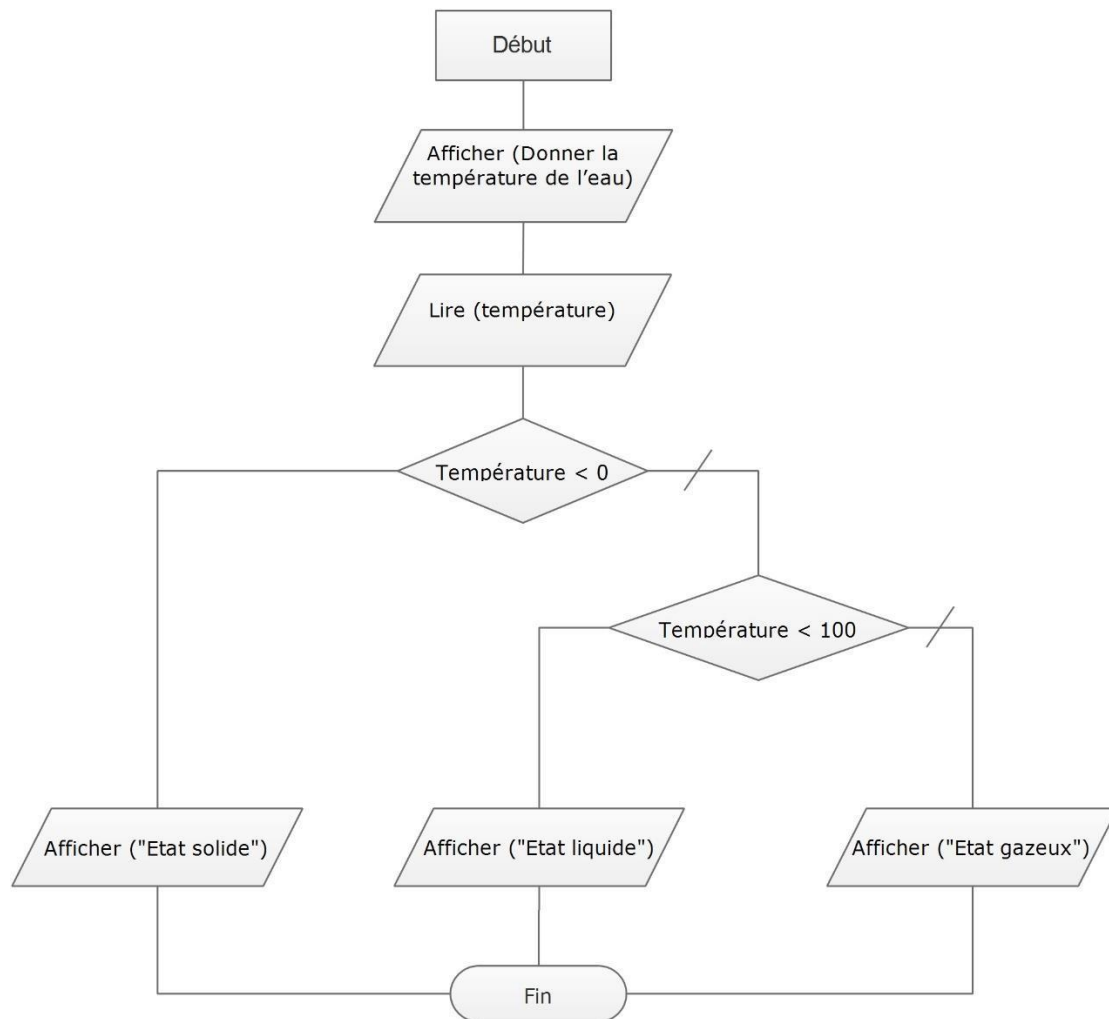


Figure 6. Organigramme « Etat de l'eau ».

Donc, l'utilisation de tests imbriqués permet de :

- Simplifier le (pseudo-) code : à travers l'imbrication nous n'avons utilisé que deux conditions simples au lieu de trois conditions dont une est composée.
➔ un algorithme (ou programme) plus simple et plus lisible.
- Optimiser le temps d'exécution : dans le cas où la première condition est vérifiée, l'algorithme passe directement à la fin, sans tester le reste qui est forcément faux.
➔ un algorithme (ou programme) plus performant à l'exécution.

Remarque :

Nous avons les équivalences suivantes :

$$\text{NON (A ET B)} \Leftrightarrow \text{NON A OU NON B}$$

$\text{NON (A OU B)} \Leftrightarrow \text{NON A ET NON B}$

Ainsi, toute structure de test avec l'opérateur logique ET peut être exprimée d'une manière équivalente avec l'opérateur logique OU et vice-versa. Par conséquent, les deux alternatives suivantes sont équivalentes [7].

Si A ET B Alors instruction(s) 2 ; Sinon instruction(s) 2 ; Finsi	↔	Si NON A OU NON B Alors instruction(s) 1 ; Sinon instruction(s) 1 ; Finsi
---	----------	---

4. Les choix multiples

Il existe une autre variante d'instructions conditionnelles qui permet d'effectuer des actions différentes suivant les différentes valeurs que peut avoir une variable. Cette structure est décrite comme suit :

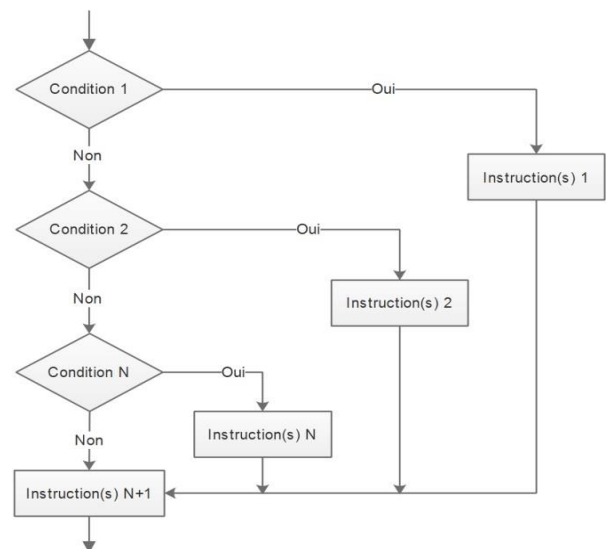
Syntaxe :

Selon (variable) valeur1 : instruction(s) 1 ;
 valeur2 : instruction(s) 2 ;
 ...
 valeurN : instruction(s) N ; **défaut :**
 instruction(s) par défaut; **FinSelon** ;

Remarque :

Dans la structure de test à choix multiples :

- L'action peut être une suite d'instructions ;
- La valeur est une constante de même type que la variable ;
- La partie « défaut » est exécutée si aucun des autres cas n'est vérifié ;
- L'exécution des différents cas (y compris le cas par défaut) est exclusive c'est-à-dire l'exécution d'un seul cas provoque la sortie de cette structure.



Exemple :

Dans ce qui suit, le nom du jour de la semaine correspondant est affiché selon la valeur de la variable « jour ».

jour ← 5 ; **Selon**

jour

1 : **Ecrire** ("Dimanche") ;

2 : **Ecrire** ("Lundi") ;

3 : **Ecrire** ("Mardi") ;

4 : **Ecrire** ("Mercredi") ;

5 : **Ecrire** ("Jeudi") ;

6 : **Ecrire** ("Vendredi") ;

7 : **Ecrire** ("Samedi") ;

Défaut : Ecrire ("Numéro de jour invalide.") ; **FinSelon**

Donc, l'expression « Jeudi » est affichée dans ce cas.

5. Conclusion

Dans ce chapitre, nous avons présenté le principe de la condition, suivant lequel un algorithme peut effectuer une action ou prendre une décision. Ceci est mis en œuvre à travers les instructions conditionnelles ou tout simplement les tests avec leurs différentes formes vues précédemment.

Chapitre quatrième : Les instructions itératives (les boucles)

1. Introduction

Considérons le même exemple qu'on a vu précédemment concernant le calcul de la moyenne générale d'un étudiant. Pour se faire, on doit :

- Lire toutes les notes (et leurs coefficients) de l'étudiant,
- Calculer la somme des notes,
- Diviser la somme obtenue sur le nombre (ou sur la somme des coefficients).

Si l'on veut maintenant calculer la moyenne d'un autre étudiant, les mêmes instructions doivent être répétées.

Pour N d'étudiants, il nous faudra donc répéter N fois la même séquence d'instructions.

Cet exemple soulève deux questions importantes :

- 1- Comment éviter d'écrire plusieurs fois la même séquence d'instructions ?
- 2- Combien de fois doit-on répéter l'exécution de la séquence d'instructions pour obtenir le résultat attendu ?

Pour répondre à ces questions, de nouvelles instructions de contrôle sont introduites. Il s'agit des instructions itératives (appelées aussi les boucles ou les itérations).

2. Définition

Une boucle (ou itération) est une instruction de contrôle qui permet de répéter plusieurs fois un ensemble d'instructions. Généralement, deux cas sont distingués :

- Le nombre de répétitions est connu.
- Le nombre des répétitions est inconnu ou variable.

3. L'instruction « Pour »

Lorsque le nombre de répétitions est déterminé (connu), l'utilisation de l'instruction « Pour » est privilégiée. Une structure de boucle avec l'instruction « Pour » s'arrête une fois que le nombre de répétitions est atteint. Cette structure possède un indice (compteur) de contrôle d'itérations caractérisé par :

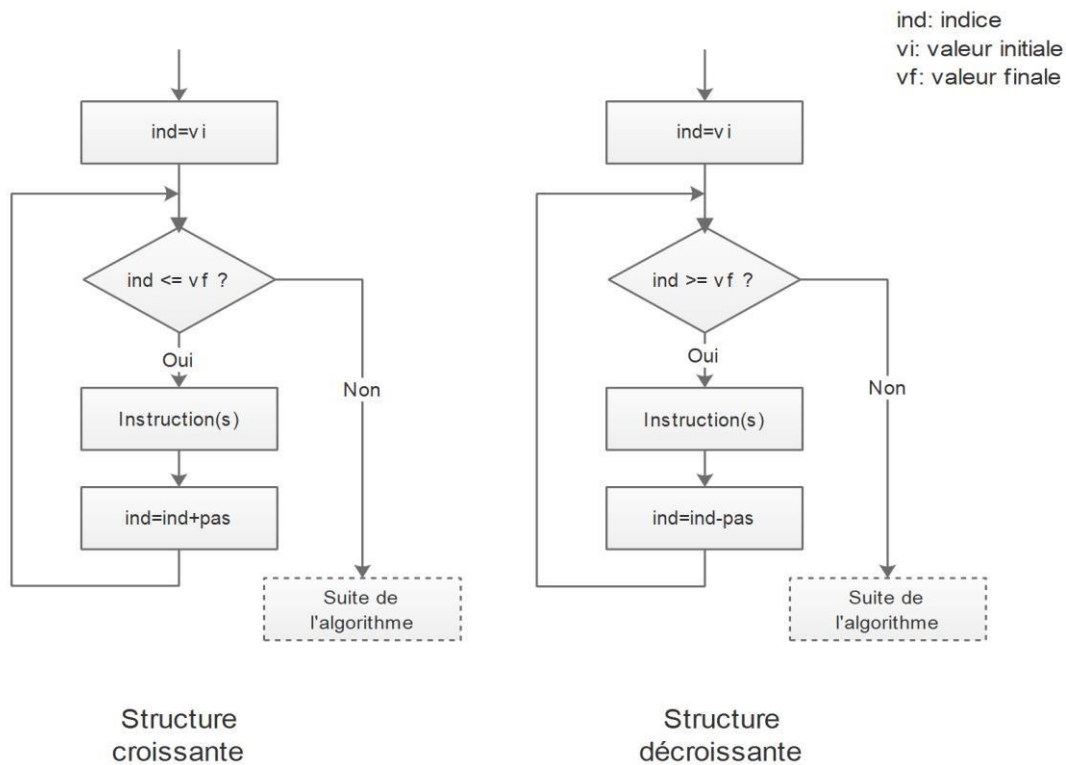
- une valeur initiale,
- une valeur finale,
- un pas de variation.

Syntaxe :

Pour indice de début à fin **Pas** valeur_du_pas

instruction(s) ;

FinPour



Cette structure est dite « croissante » lorsque la valeur initiale de l'indice est inférieure à sa valeur finale, le pas de variation est par conséquent positif. Autrement, elle est dite « décroissante ».

Exemple : un compteur croissant/décroissant

Les deux algorithmes suivants comptent de 1 à N et de N à 1 respectivement.

Algorithme compteur_croissant ; **Algorithme**
Var i : entier ; **Var** i : entier ;
Const N=100 ; **Const** N=100 ;
Début **Début**
Pour i **de** 1 **à** N /* par défaut le pas = 1 */
Ecrire (i) ; **Ecrire** (i) ;
FinPour **FinPour**
Fin **Fin**

compteur_decroissant ;
 /* **Pour** i **de** N **à** 1 /* par défaut le pas = -1

Résultat d'exécution : 1, 2, 3, ..., 99, 100 Résultat d'exécution : 100, 99, 98, ..., 2, 1

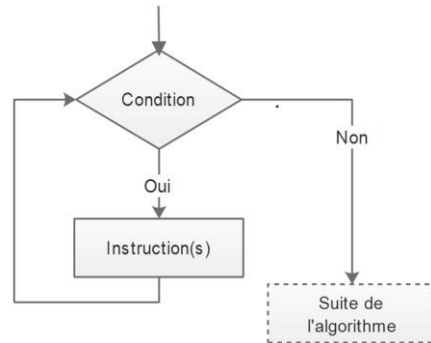
Remarque : Si la valeur du « pas » n'est pas précisée dans l'instruction « Pour », elle est par défaut égale à un (1).

4. L'instruction « Tant que... faire »

Cette instruction permet de tester une condition et répéter le traitement associé tant que cette condition est vérifiée.

Syntaxe:

Tant que condition **faire**
instruction(s) ;
FinTq



Exemple : Réécrivons l'algorithme précédent avec cette instruction.

```

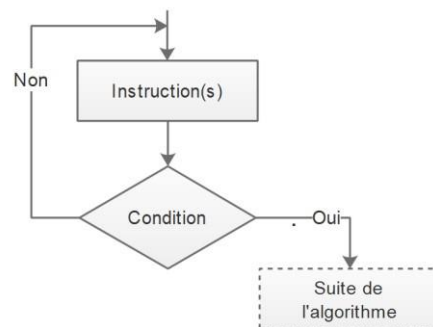
Var i : entier ;
Début
    i ← 1 ;
    Tant que (i<=100) faire
        Ecrire (i) ; i ← i+1 ;
    FinTq
Fin
    
```

6. L'instruction « Répéter... jusqu'à »

Dans cette instruction, un traitement est exécuté au moins une fois puis sa répétition se poursuit jusqu'à ce que la condition soit vérifiée.

Syntaxe:

Répéter
instruction(s) ;
Jusqu'à (condition) ;



Exemple : Soit l'algorithme suivant :

```

Var n, p : entier ;
Début
    Répéter
        Ecrire ("Donner un nombre :) ; Lire (n) ; p
        ← n*n ; Ecrire (p);
    Jusqu'à (n=0)
    Ecrire ("Fin de l'algorithme") ;
Fin
    
```

Les instructions encadrées par les mots **répéter** et **jusqu'à** constituent le bloc de la boucle qu'il faut répéter jusqu'à ce que la condition (**n=0**) soit vérifiée. Donc le nombre de répétitions de cette boucle dépend des données fournies par l'utilisateur.

Question ?

Réécrire l'algorithme précédent avec « Tant que... faire » puis avec « Pour ».

Remarque :

Dans la boucle « Répéter... jusqu'à », la condition telle qu'elle est exprimée ci-dessus, constitue une **condition d'arrêt** de la boucle ; mais réellement, cela diffère selon le langage de programmation utilisé. Par exemple, en Pascal, la condition de cette boucle est une condition d'arrêt. Alors qu'en langage C, cette condition est exprimée en tant qu'une **condition de continuation**.

7. La notion du compteur

Un compteur est une variable associée à la boucle dont la valeur est incrémentée de un à chaque itération. Elle sert donc à compter le nombre d'itérations (répétitions) de la boucle. La notion du compteur est associée particulièrement aux deux boucles : «

Répéter...jusqu'à » et « **Tant que...faire** ». Par contre, dans la boucle « **Pour** », c'est l'indice qui joue le rôle du compteur.

L'utilisation du compteur dans les deux premières boucles est exprimée ainsi :

compt ← 0 ;	compt ← 0 ;
Répéter	Tant que (condition) faire
instruction(s) ; instruction(s) ;	
<i>Bloc de la boucle</i>	
compt ← compt +1 ; compt ← compt +1 ; Jusqu'à (condition) ; FinTant que ;	

Remarque :

Il faut toujours initialiser le compteur avant de commencer le comptage. La variable « compt » (utilisée ci-dessus comme compteur), a été initialisée à zéro (0) avant le début de chaque boucle.

L'instruction « compt ← compt +1 » incrémente la valeur de « compt » de un (1). Elle peut être placée n'importe où à l'intérieur du bloc de la boucle.

Exemple :

$i \leftarrow 0 ; i \leftarrow 0 ;$ Répéter Tant que ($i < 5$) faire Ecrire (i); Ecrire (i); $i \leftarrow i + 1 ; i \leftarrow i$ Jusqu'à ($i = 5$) ; FinTant que ; Résultat d'exécution : 0, 1, 2, 3, 4	+1 ; d'exécution : 0, 1, 2, 3, 4
--	--

8. La notion d'accumulation

Cette notion est fondamentale en programmation. Elle est utilisée notamment pour calculer la somme d'un ensemble de valeurs. L'instruction correspondante se présente ainsi :

$\text{variable} \leftarrow \text{variable} + \text{valeur} ;$

Cette instruction consiste à ajouter une valeur à une variable numérique, puis affecter le résultat dans la variable elle-même. En d'autres termes, la nouvelle valeur de variable égale à l'ancienne plus une certaine valeur [4].

Exemple : calcul de la somme de n valeurs données par l'utilisateur :

```

Var i, n : entier ; som, val : réel ;
Début
    Écrire ("Donner le nombre de valeurs :") ; Lire (n) ; som
     $\leftarrow 0 ;$ 
    Pour i de 1 à n
        Écrire ("Enter une valeur :") ; Lire (val) ;
         $\text{som} \leftarrow \text{som} + \text{val} ;$ 
    Finpour
    Écrire ("La somme des valeurs est égale à :", som) ; Fin
    
```

9. Les boucles imbriquées

Les boucles peuvent être imbriquées les unes dans les autres. Deux ou plusieurs boucles imbriquées peuvent être aussi les mêmes ou différentes.

Exemple :

Pour i de 1 à 2 Écrire ("i = ", i) ; Pour j de 1 à 3 Écrire ("j = ", j) ; Finpour Finpour	} } } } }	<i>boucle 1</i> <i>boucle 2</i>
--	--	---

Dans l'exemple ci-dessus, chaque itération de la boucle extérieure (boucle 1) exécute la boucle intérieure (boucle 2) jusqu'à la fin avant de passer à l'itération suivante, et ainsi de

suite jusqu'à la fin des deux boucles. Ainsi, le résultat d'exécution peut être représenté comme suit :

```
i = 1 j
= 1 j
= 2 j
= 3 i
= 2 j
= 1 j
= 2 j
= 3
```

Remarque :

Des boucles peuvent être imbriquées ou successives. Cependant, elles ne peuvent jamais être croisées. Par exemple, l'algorithme suivant est faux puisqu'il comporte deux boucles croisées :

```
Var i, j : entier ;
Début
  i ← 1 ; j ← 1 ;
  Répéter Écrire
    i ;
    Répéter
      Écrire j ;
    i ← i + 1 ;
  Jusqu'à i > 2
    j ← j + 1 ;
  Jusqu'à j > 3
Fin
```

10. Conclusion

Ce chapitre a été consacré aux structures itératives ou boucles qui permettent de répéter l'exécution d'une séquence d'instructions plusieurs fois selon un nombre fixe ou certains critères dont l'utilisation a été explicitée à travers différents exemples. Ainsi, ces instructions sont d'une grande importance dans la manipulation de certaines structures de données telles que les tableaux que nous aborderons dans le prochain chapitre.

Chapitre cinquième : Les tableaux

1. Introduction

Supposons que l'on a besoin de stocker et de manipuler les notes de 100 étudiants. On doit, par conséquent, déclarer 100 variables : $n_1, n_2, \dots, n_{100}$. Vous pouvez remarquer que c'est un peu lourd de manipuler une centaine de variables (avec 100 fois de lecture/écriture...). Imaginons maintenant le cas pour une promotion de 1000 étudiants, alors là devient notre cas un vrai problème.

En algorithmique (et en programmation), on peut regrouper toutes ces variables en une seule structure qui s'appelle **tableau**.

Un tableau est un ensemble de variables de même type ayant toutes le même nom.

Suite à cette définition, la question suivante se pose :

- Comment peut-on différencier entre des variables ayant le même nom ?

La réponse est dans la notion du tableau lui-même où chaque élément est repéré par un **indice**. Ce dernier est un numéro (généralement un entier) qui permet de différencier chaque élément du tableau des autres. Ainsi, les éléments du tableau ont tous le même nom, mais pas le même indice. Pour accéder à un élément d'un tableau, on utilise le nom du tableau suivi de l'indice de l'élément entre crochets [4].

Exemple

Soit le tableau T contenant les valeurs suivantes : 5, 10, 29, 3, 18 et 14 : L'organisation du tableau T dans la mémoire peut être représentée comme suit :

T 	<i>indices :</i>	i=0	i=1	i=2	i=3	i=4	i=5
	<i>valeurs :</i>	T[0]=5	T[1]=10	T[2]=29	T[3]=3	T[4]=18	T[5]=14

2. Tableaux à une seule dimension

Dans ce type de tableaux, chaque élément est accessible (pour lecture ou modification) par un seul indice.

2.1. Déclaration

La syntaxe de déclaration d'un tableau à une seule dimension est la suivante :

Tableau nom_du_tableau [taille] : **type** ;

Exemple : Tableau Notes [100] : réel ;

Remarque :

L'indice d'un élément dans un tableau, peut être exprimé comme un nombre, mais aussi il peut être exprimé comme une variable ou une expression calculée [10].

La valeur de l'indice doit être toujours :

- **Supérieur ou égal à 0** : dans quelques langages, le premier élément d'un tableau porte l'indice 1 (comme en Pascal). Mais dans d'autres, comme c'est le cas en langage C, la numérotation des indices commence à zéro. Par exemple *Notes[1]* est le deuxième élément du tableau *Notes*.
- **de type entier** : quel que soit le langage, l'élément *Notes[1, ...]* n'existe jamais.
- **Inférieur ou égal au nombre des éléments du tableau (moins 1 si l'on commence à zéro)**: En langage C, si un tableau *T* est déclaré comme ayant 10 éléments, la présence, dans une ligne du corps de l'algorithme, de l'expression *T[10]* déclenchera automatiquement une erreur.

2.2. Manipulation

Une fois déclaré, un tableau peut être manipulé comme un ensemble de variables simples.

Les trois manipulations de base sont l'affectation, la lecture et l'écriture [4].

2.2.1. L'affectation

L'affectation d'une valeur *v* à un élément *i* d'un tableau *T* de type numérique, se fait par :

$T[i] \leftarrow v$;

Par exemple, l'instruction : $T[0] \leftarrow 5$; affecte la valeur 5 au premier élément du tableau *T*.

Illustration :

	indices :	i=0	i=1	i=2	i=3	i=4	i=5
T	→valeurs :	5					

Supposons maintenant

que l'on veut affecter la même valeur à tous les éléments du tableau *T*, on utilisera pour cela une boucle :

Pour *i* de 0 à *n*-1 // *n* est la taille de *T*
 $T[i] \leftarrow 5$;
FinPour

Cette boucle permet de parcourir tout le tableau *T* en affectant à chaque élément la valeur 5.

Illustration :

	<i>indices :</i>	i=0	i=1	i=2	i=3	i=4	i=5
T	→	<i>valeurs :</i>					
		5	5	5	5	5	5

2.2.2. La lecture

Il est possible aussi d'affecter des valeurs aux éléments d'un tableau par une instruction de lecture.

Exemple :

Ecrire "Entrer une note :";

Lire T[0];

Dans cet exemple, la valeur saisie est affectée au premier (1^{er}) élément du tableau T.

Illustration :

	<i>indices :</i>	i=0	i=1	i=2	i=3	i=4	i=5
T	→	<i>valeurs :</i>					
		0					

2.2.3. L'écriture

De même que la lecture, l'écriture de la valeur d'un élément du tableau s'écrit comme suit :

Ecrire T[i]; Cette instruction permet d'afficher la valeur de l'élément i du tableau T.

Remarque :

Les éléments d'un tableau sont manipulés de la même façon que les variables simples. S'il s'agit d'un tableau de type numérique, les éléments peuvent être utilisés dans l'évaluation des expressions numériques du genre :

$x \leftarrow (\text{Notes}[1] + \text{Notes}[2]) / 2$; Notes

$[0] \leftarrow \text{Notes}[0] + 1$;

Exemple :

Soit *Notes* un tableau de valeurs réelles tel qu'il est illustré dans le schéma qui suit.

Illustration :

	<i>indices :</i>	i=0	i=1	i=2	i=3	i=4	i=5
Notes	→	<i>valeurs :</i>					
		10	15	7	8.25	11.5	16

L'exécution de l'instruction : $x \leftarrow (\text{Notes}[1] + \text{Notes}[2]) / 2$;

Est équivalente à l'opération : $x = \frac{15+7}{2} = 11$

2.3. Application 1

Ecrivons un algorithme qui permet de lire des valeurs saisies par l'utilisateur dans un tableau de taille 20, et les afficher par la suite.

Algorithme :

```

Var i : entier ;
Tableau Tab [20] : réel ;
Début
    Pour i de 0 à 19
        Ecrire ("Donner une valeur :") ;
        Lire (Tab[i]) ;
    FinPour
    Pour i de 0 à 19
        Ecrire ("Valeur ", i, "=", Tab[i]) ;
    FinPour
Fin
    
```

L'exécution du programme correspondant à l'algorithme ci-dessus, permet d'initialiser le tableau Tab comme suit :

- Après la fin de la 1^{ère} boucle (boucle de lecture) :

		<i>indices :</i>						
		i=0	i=1	...	i=18	i=19		
Tab								
	<i>valeurs :</i>	5	34	...	14.5	60		

- Après la fin de la 2^{ème} boucle (boucle d'affichage), on aura, sur écran par exemple, l'affichage suivant :

```

Valeur 0 = 5
Valeur 1 = 34
...
Valeur 18 = 14,5
Valeur 19 = 60
    
```

Remarque : les valeurs : 5, 34, 14.5 et 60 sont un exemple d'échantillon de valeurs saisies par l'utilisateur.

3. Tableaux à deux dimensions

Les tableaux à deux dimensions se présentent généralement sous forme d'un ensemble de lignes et de colonnes (Matrice). Par conséquent, chaque élément est repéré par deux indices.

Exemple : le tableau T ci-dessous possède 3 lignes et 4 colonnes.

	j=0	j=1	j=2	j=3	
i=0					i : indice des lignes
i=1					j : indice des colonnes
i=2		x			

Donc, T[2][1] (i=2 et j=1) désigne l'élément de la 3^{ème} ligne et la 2^{ème} colonne (en commençant de zéro bien sûr).

3.1. Déclaration d'un tableau à deux dimensions

Syntaxe :

Tableau nom_tableau [taille1][taille2] : **type** ;

Exemple : **Tableau** Notes [10][20] : **réel** ;

Le tableau Notes est composé de 10 lignes et 20 colonnes. Ce tableau pourra contenir donc 10*20 soit 200 valeurs réelles.

Remarque :

L'utilité d'un tableau à deux dimensions réside dans la possibilité de déclarer un seul tableau au lieu de déclarer plusieurs tableaux identiques. En effet, le tableau de l'exemple précédent est équivalent à 10 tableaux simples de 20 éléments chacun. En d'autres termes, la déclaration :

Tableau Notes [10][20] : **réel** ; remplace celle-ci :

Tableau Notes1 [20], Notes2 [20],..., Notes10 [20] : **réel** ;

3.2. Manipulation d'un tableau à deux dimensions

Un tableau à deux dimensions est manipulé de la même façon qu'un tableau simple (à une seule dimension) que ce soit pour l'affectation, la lecture ou l'écriture. Ces trois opérations sont illustrées dans la sous-section suivante.

3.3. Application 2

Reprenons l'algorithme de la sous-section 2.3 mais cette fois-ci pour un tableau de 5 lignes et 20 colonnes :

```
Var i, j : Entier ;
Tableau Tab [5][20] : réel ;
Début
    Pour i de 0 à 4
        Pour j de 0 à 19
            Ecrire ("Donner une valeur :") ;    Lire (Tab [i][j]) ; Finpour
        Finpour
    Pour i de 0 à 4
        Pour j de 0 à 19
            Ecrire (Tab [i][j]) ;
        Finpour
    Finpour Fin
```

4. Tableaux à n dimensions

Les tableaux à n dimensions ($n > 2$), peuvent être utilisés pour diverses raisons telles que la création et le traitement des objets 3D par exemple qui nécessitent des tableaux de 3 dimensions au minimum. La déclaration de ce type de tableaux est comme suit :

Syntaxe :

```
Tableau nom_tableau [taille1][taille2] ... [tailleN] : type ;
```

Exemple : Tableau T [10][20][50] : réel ; // un tableau T à 3 dimensions

La manipulation d'un tableau à plusieurs dimensions suit le même principe que celle des tableaux à deux dimensions. Ceci s'appuie sur l'utilisation des boucles imbriquées pour parcourir le tableau, de sorte qu'il y aura autant de boucles qu'il y a de dimensions.

5. La recherche dans un tableau

5.1. La notion du drapeau

Le drapeau (ou *flag* en Anglais) est une variable booléenne initialisée à Faux (drapeau baissé). Dès qu'un évènement attendu se produit, la variable change de valeur à Vrai (drapeau levé). Donc, la valeur finale du drapeau permet de savoir si un évènement a eu lieu ou pas. Cela devrait s'éclairer à l'aide d'un exemple extrêmement fréquent : la recherche de l'occurrence d'une valeur dans un tableau [10]. **Exemple :**

En supposant l'existence d'un tableau comportant N valeurs entières. On doit écrire un algorithme qui lit un nombre donné et informe l'utilisateur de la présence ou de l'absence de ce nombre dans le tableau. La première étape consiste à écrire les instructions de lecture du nombre N et de parcours du tableau :

```
Tableau Tab[N] : Entier ;  
Var val, i : Entier ;  
Début  
    Ecrire ("Entrer la valeur à rechercher :") ; Lire (val) ;  
    Pour i de 0 à N-1  
        ...  
    Finpour  
Fin
```

Illustration :

On suppose que N=6 et les valeurs saisies sont celles figurant dans le schéma suivant :

<i>indices :</i>	i=0	i=1	i=2	i=3	i=4	i=5
Tab 	10	22	31	46	5	7

Revenons à l'algorithme, maintenant, il faut combler les points de la boucle (le bloc qui devra contenir les instructions de la recherche). Évidemment, il va falloir comparer « val » à chaque élément du tableau, s'il y a une égalité quelque part, alors « val » fait partie du tableau. Cela va se traduire, bien entendu, par l'instruction conditionnelle « Si ... Alors ... Sinon ».

```
...  
Début  
    Ecrire ("Entrez la valeur à rechercher ") ; Lire (val) ;  
    Pour i de 0 à N-1
```

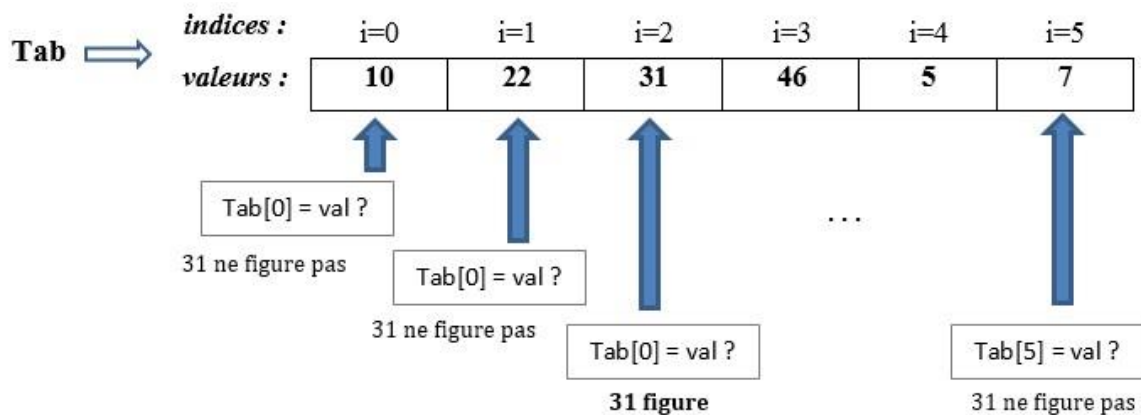
```

    Si (val = Tab[i]) Alors
        Ecrire (val, "figure") ; ?
    Sinon
        Ecrire (val, "ne figure pas") ;
    Finsi
Finpour Fin

```

Illustration :

On suppose que la valeur à rechercher (val) est égale à 31 :



On peut constater qu'on a deux possibilités :

- ou bien la valeur « val » figure dans le tableau, - ou bien elle n'y figure pas.

Mais dans tous les cas, l'algorithme ne doit produire qu'une seule réponse, quel que soit le nombre d'éléments du tableau. Or, l'algorithme ci-dessus affiche autant de messages qu'il y a de valeurs dans le tableau. Il y a donc une **erreur** quelque part.

En fait, on ne peut savoir si la valeur recherchée existe dans le tableau ou non que lorsque le parcours du tableau est entièrement accompli.

Pour pallier à cette erreur, on doit réécrire l'algorithme en plaçant le test après la boucle et en utilisant cette fois-ci une variable booléenne (drapeau) que l'on appelle « Existe ».

Cette variable doit être gérée comme suit :

- La valeur de départ de « Existe » doit être évidemment **Faux** (drapeau baissé).

- La valeur de la variable « Existe » doit devenir **Vrai** (drapeau levé), si un test dans la boucle est vérifié (lorsque la valeur de « val » est rencontrée dans le tableau). mais le test doit être asymétrique, c.à.d. qu'il ne comporte pas de "sinon".

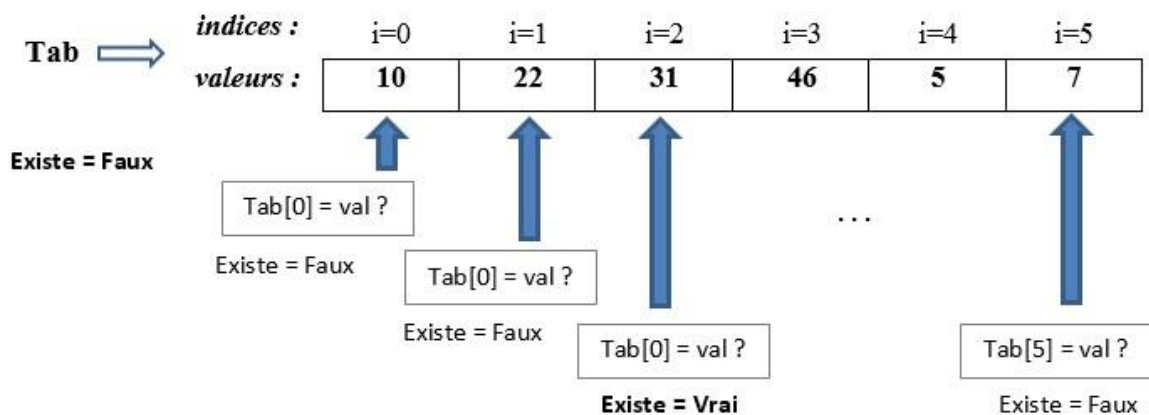
Donc l'algorithme correct devrait être comme suit :

```

Tableau Tab[N] : entier ;
Var val, i : entier ;
existe : booléen ;
Début
    Ecrire ("Entrez la valeur à rechercher ") ;
    Lire (val) ; existe
    ← faux ; Pour i de
    0 à N-1
        Si (val = Tab[i]) Alors existe ← vrai; Finsi Finpour
    Si (existe) Alors Ecrire (val, "fait partie du tableau") ;
    Sinon Ecrire (val, "ne fait pas partie du tableau") ;
Finsi Fin
    
```

Illustration :

En utilisant un drapeau (la variable « Existe ») :



□ Question ?

Réécrire le même algorithme mais cette fois-ci, la boucle de recherche doit être arrêtée dès que la valeur du drapeau change.

6. Le tri d'un tableau

Qu'est-ce qu'un tri ?

Le tri est une opération consistant à ordonner un ensemble d'éléments suivant une relation d'ordre prédéfinie. Le problème du tri est un grand classique en algorithmique. Trier un tableau numérique c'est donc ranger ses éléments en ordre croissant ou décroissant. Il

existe plusieurs algorithmes de tri, parmi lesquels le tri par sélection, tri par insertion, tri à bulles, tri par fusion, etc. Nous illustrons dans ce qui suit deux types de tri, à savoir le tri par sélection et le tri par insertion.

6.1. Tri par sélection

Cette technique est parmi les plus simples, elle consiste à sélectionner, pour une place donnée, l'élément qui doit y être positionné. Par exemple pour trier un tableau en ordre croissant, on met en première position le plus petit élément du tableau et on passe à la position suivante pour mettre le plus petit élément parmi les éléments restants et ainsi de suite jusqu'au dernier [2].

Exemple :

Soit à trier, en ordre croissant, le tableau suivant :

25	10	13	31	22	4	2	18
----	----	----	----	----	---	----------	----

Nous commençons par la recherche de la plus petite valeur et sa position. Une fois identifiée

(dans ce cas, c'est le nombre 2 en 7^{ème} position), nous l'échangeons avec le 1^{er} élément (le nombre 25). Le tableau devient ainsi :

2	10	13	31	22	4	25	18
----------	----	----	----	----	---	-----------	----

Nous recommençons la recherche, mais cette fois, à partir du 2^{ème} élément (puisque le 1^{er} est à sa position correcte). Le plus petit élément se trouve en 6^{ème} position (le nombre 4).

Nous échangeons donc le 2^{ème} élément avec le 6^{ème} élément :

2	4	13	31	22	10	25	18
---	----------	----	----	----	-----------	----	----

Nous recommençons la recherche à partir du 3^{ème} élément (puisque les deux premiers sont maintenant bien placés), Le plus petit élément se trouve aussi en 6^{ème} position (10), en l'échangeant avec le 3^{ème}, ça donnera:

2	4	10	31	22	13	25	18
---	---	-----------	----	----	-----------	----	----

Nous recommençons maintenant à partir du 4^{ème} élément et de la même façon nous procédons jusqu'à l'avant dernier :

2	4	10	13	22	31	25	18
---	---	----	-----------	----	-----------	----	-----------

2	4	10	13	18	31	25	22
---	---	----	----	-----------	----	----	-----------

2	4	10	13	18	22	25	31
---	---	----	----	----	-----------	-----------	-----------

2	4	10	13	18	22	25	31
---	---	----	----	----	----	----	----

Algorithmiquement, nous pouvons décrire ce processus de la manière suivante :

- Boucle principale : prenant comme point de départ le premier élément, puis le second, etc, jusqu'à l'avant dernier.
- Boucle secondaire : à partir de ce point de départ mouvant, nous recherchons jusqu'à la fin du tableau le plus petit élément. Une fois trouvé, nous l'échangeons avec le point de départ.

Donc, cela s'écrit :

/ boucle principale : le point de départ se décale à chaque tour */*

Pour i de 0 à 6

/ on considère provisoirement que T(i) est le plus petit élément */*

posmin ← i ; // posmin est la position du minimum initialisée par i

/ on examine tous les éléments suivants */*

Pour j de (i + 1) à 7

Si (T(j) < T(posmin)) **Alors** posmin ← j ; **Finsi**

Finpour

/ on sait maintenant où est le plus petit élément. Il ne reste plus qu'à effectuer la permutation */*

temp ← T(posmin) ;

T(posmin) ← T(i) ;

T(i) ← temp ;

/ On a placé correctement l'élément numéro i, on passe à présent au suivant */*

FinPour

6.2. Tri par insertion

Soit à trier un tableau d'éléments en ordre croissant.

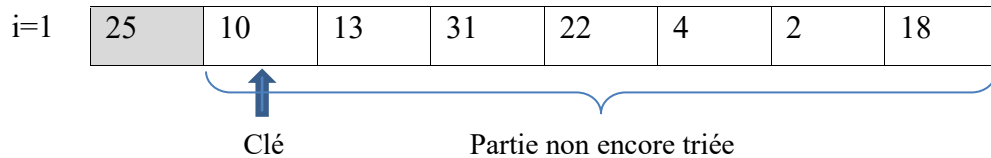
Le principe de ce type de tri repose à chaque itération sur trois phases [2] :

- On prend le premier élément dans la partie non encore triée du tableau (la clé).
- On cherche la place de la clé dans la partie déjà triée du tableau, en commençant par la droite de cette partie.
- Une fois cette place trouvée, on y insère la clé après qu'on ait décalé vers la droite tous les éléments de la partie triée dont la valeur est plus grande ou égale à la valeur de la clé.

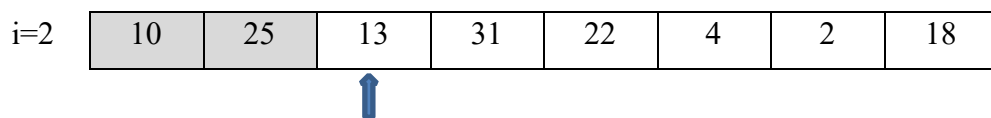
Il faut noter qu'initialement, la partie triée est constituée seulement du premier élément du tableau, autrement dit, le processus du tri commence à partir du deuxième élément.

Exemple :

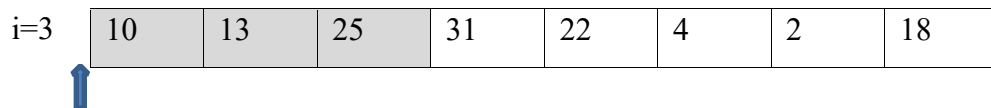
Soit à trier, en ordre croissant, le même tableau précédent en appliquant le tri par insertion :



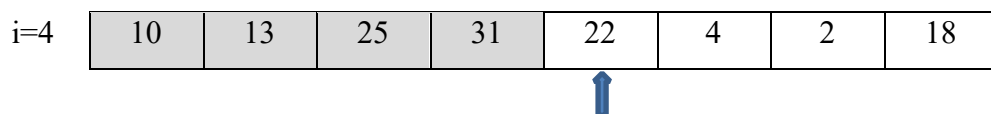
On décale le 1^{er} élément de la partie triée vers la droite puisque sa valeur est supérieure à la clé. Cette dernière est déplacée à la 1^{ère} position :



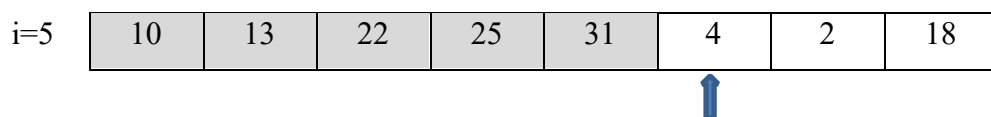
On recommence le processus avec une nouvelle clé. Le 1^{er} élément à droite de la partie triée (25) est décalé vers la droite puisque sa valeur est supérieure à la clé. Le 2^{ème} élément ne sera pas décalé puisqu'il est inférieur à la clé. Par conséquent, la clé est insérée dans la 2^{ème} position du tableau :



On ne déplace pas cette clé (31) puisque sa valeur est supérieure à celles des éléments qui la précèdent.



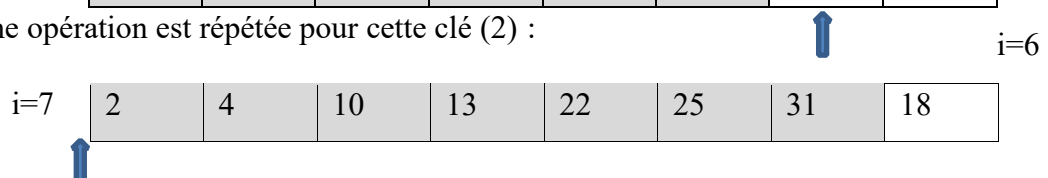
On décale les deux premiers éléments (31 et 25) vers la droite et la clé est insérée à la 3^{ème} position :



On décale tous les éléments de la partie triée vers la droite puisque leurs valeurs sont supérieures à celle de la clé. Cette dernière est déplacée à la 1^{ère} position :



La même opération est répétée pour cette clé (2) :



Les trois éléments (22,25 et 31) sont décalés vers la droite et la clé est déplacée vers la 5^{ème} position :

2	4	10	13	18	22	25	31
---	---	----	----	----	----	----	----

Nous obtenons donc un tableau qui est trié en ordre croissant.

L'algorithme correspondant à ce type de tri est présenté dans ce qui suit :

Algorithme Tri_Insertion ;

Var i, j, n, clé : **Entier**; // n est la taille du tableau T

T : **Tableau d'entiers** ;

Début

Pour i de 1 à n-1 // on commence par le 2^{ème} élément (début de la partie non triée) clé

□ T[i] ;

j □ i - 1 ; // indice du 1^{er} élément à droite de la partie triée

Tant que ((j >= 0) **ET** (clé < T[j])) **Faire**

T[j + 1] □ T[j]; // Décalage j □ j

- 1;

FinTant que

T[j + 1] □ clé; // Insertion de la clé

FinPour

Fin

6.3. Comparaison

Dans l'algorithme de tri par sélection, nous avons dans tous les cas, la boucle interne est exécuté pour i=1, 2, 3 jusqu'à i=(n-1) par conséquent, nous avons (n-1) + (n-2) + (n-3) + ... + 1 étant $n(n-1)/2$ exécutions. Par exemple, pour un tableau de 100 éléments, la boucle est exécutée 4950 fois dans tous les cas.

Dans l'algorithme de tri par insertion, nous avons dans le pire des cas un tableau trié à l'envers (en ordre décroissant dans ce cas), et la boucle interne est exécuté (n-1) + (n-2) + (n-3) + ... + 1 fois, étant $n(n-1)/2$ exécutions au maximum. Au meilleur des cas, le tableau est trié en ordre voulu (croissant dans ce cas) et la boucle interne ne s'exécutera jamais. En moyenne, le nombre d'exécutions est $n(n-1)/4$. Par exemple, pour un tableau de 100 éléments, la boucle est exécutée 4950 fois au maximum et 2475 en moyenne.

Chapitre sixième : Les fonctions et les procédures

1. Introduction

La fiabilité, la lisibilité et la réutilisabilité des programmes, reposent sur l'utilisation des sous-programmes. Ces derniers permettent :

- La réduction de la taille des programmes : il est possible de déterminer les blocs analogues, les substituer par un sous-programme, ensuite l'appeler dans des points déterminés au niveau du programme principal.
- L'organisation du code : le problème initial peut être découpé en sous-problèmes (modules) où chacun sera résolu par un sous-programme [1].

Exemple : Codage en base b [3]

Un entier positif en base b est représenté par une suite de chiffres $(c_n c_{n-1} \dots c_1 c_0)_b$ où les c_i sont des chiffres de la base b ($0 \leq c_i < b$).

L'équivalent décimal de ce nombre est :

$$c_n b_n + c_{n-1} b_{n-1} + \dots + c_1 b_1 + c_0 b_0 = \sum_{i=0}^n c_i b_i$$

On suppose que le nombre x est représenté par un tableau de chiffres (code) en base b ; par exemple si $b = 2$ et $\text{code} = [1, 0, 1]$, alors en base 10 le nombre entier x correspondant vaudra

$1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 4 + 0 + 1 = 5$. Etant donné code et b, l'algorithme qui permet de calculer x en base 10 est le suivant :

```
x ← 0 ;  
Pour i de 0 à L-1 // L est la longueur du code x  
  ← x + code[i]*b^(L-1-i);
```

Supposons que l'on veut calculer successivement la valeur décimale x des nombres $(123)_5$ et $(123)_8$, on devra donc recopier deux fois l'algorithme ci-dessus.

```
b ← 5 ;  
[1,2,3] ;  
;  
b ← 8 ; code ←  
code ← [1,2,3] ; x ← 0  
x ← 0 ;
```

Pour i de 0 à L-1

code[i]*b^(l-1-i);

Pour i de 0 à L-1 x ← x +

x ← x + code[i]*b^(l-1-i);

Dans la pratique, il n'est pas souhaitable d'écrire deux fois le même programme, d'autant plus, si celui-ci nécessite de très nombreuses lignes de code source. Pour améliorer la réutilisabilité de l'algorithme la solution est d'encapsuler le code à répéter au sein d'un sousprogramme.

2. La notion de sous-programme

Un sous-programme est une portion de code analogue à un programme, destiné à réaliser une certaine tâche à l'intérieur d'un autre programme. Il est identifié par un nom unique et un bloc d'instructions qui peut être exécuté plusieurs fois par des appels. Un appel est une instruction qui fait partie d'un autre programme ou sous-programme appelé le (sous-) programme appelant.

Pour résumer, un sous-programme est utilisé pour deux raisons essentielles :

- Lorsqu'une tâche est répétée plusieurs fois : on écrit un sous-programme pour cette tâche et l'on appelle à chaque endroit où l'on en a besoin c.à.d. on évite de réécrire le même code à plusieurs endroits dans le même programme.
- Pour réaliser la structuration d'un problème en sous-problèmes : on divise le problème en sous-problèmes pour mieux le contrôler (diviser pour régner).

Il existe deux types de sous-programmes : les **fonctions** et les **procédures**. Cependant, avant de détailler ces deux concepts, il sera utile de définir quelques notions utiles.

2.1. La portée d'une variable

Dans cette sous-section, nous illustrons une notion appelée **portée d'une variable**. L'idée principale est qu'il est possible d'avoir deux variables différentes avec le même nom toutefois qu'elles ont des portées différentes. Le cas le plus connu que l'on peut citer, est qu'une variable définie au niveau du programme principal (celui qui résout le problème initial) est appelée **variable globale** et sa portée est totale, par conséquent, tout sous-programme du programme principal peut utiliser cette variable. Alors qu'une variable définie au sein d'un sous-programme est appelée **variable locale** dont la portée est limitée au sous-programme dans lequel elle est déclarée [7].

Remarque :

Si l'identificateur (le nom) d'une variable locale est identique à celui d'une variable globale, cette dernière est localement masquée. Autrement dit, la variable globale devient inaccessible dans le sous-programme contenant la variable locale de même nom.

Exemple :

Supposons qu'une partie de notre programme sera le sous-programme suivant :

```

Algorithme Secondaire ;
Var x : entier ; // variable locale
Début x ← 3 ;
        Ecrire (x, y) ;
Fin

```

Supposons maintenant que nous appelons ce sous-programme « Secondaire » depuis une autre partie du programme « Principal » qui utilise également deux variables globales :

```

Algorithme Principal ;
Var x, y : entier ; // variables globales
Début x ← 5 ; y ← 8 ;
        ... ; // Appel au sous-programme Secondaire
        Ecrire (x, y) ; Fin

```

L'exécution de ce programme peut être illustrée comme suit :

Programme	Espace mémoire	Résultat d'affichage
-Principal Début Fin	Appel au sous-programme x = 5 y = 8	5 , 8
-Secondaire Début ... Fin	x = 3	3 , 8

Dans cet exemple, la variable « x », ayant la valeur 5 dans le programme principal, est une variable globale. Une autre variable qui porte le même nom « x » est utilisée au niveau du programme secondaire ayant comme valeur 3. Cet variable locale a masqué la variable globale « x » au niveau du programme secondaire, par conséquent, l'affichage de « x » au niveau de ce sous-programme correspond à la valeur 3. Par contre, la variable globale y est

quant à elle accessible, ce qui justifie l’affichage de la valeur 8 au niveau du sousprogramme.

2.2. Les paramètres

Les paramètres d'un sous-programme sont un ensemble de variables locales (**paramètres formels**) associées à un ensemble de variables ou constantes du (sous-) programme appelant (**paramètres effectifs**).

Remarque :

- Un paramètre est une variable locale, donc il admet un type.
- L’appel d’un sous-programme possédant un ou plusieurs paramètres, implique une association entre ces paramètres et les paramètres effectifs du programme (ou sousprogramme) appelant, respectivement de même type.

Par exemple, si le sous-programme *Racine* permet de calculer la racine carrée d'un réel :

- Ce sous-programme admet un seul paramètre de type réel positif.
- Le programme appelant *Racine* doit fournir le réel positif dont il veut calculer la racine carrée, cela peut être une variable (*Racine (y)*) ou une constante (*Racine (9)*).

2.3. Le passage de paramètres

L'association entre les paramètres effectifs et les paramètres formels est appelé passage de paramètres. Il existe deux types de passage de paramètres :

- Le passage par valeur.
- Le passage par référence (ou adresse).

Dans le premier type, la valeur du paramètre effectif est affectée (copiée) au paramètre formel correspondant. Sachant que les paramètres formels ne sont que des variables locales de la fonction. Ce type de passage sera illustré dans la section qui suit.

Dans le second type, c’est l’adresse du paramètre effectif qui est affectée au paramètre formel correspondant. Ceci rend possible au sous-programme d’accéder directement à l’adresse de la variable concernée (le paramètre effectif) et par conséquent la modifier si nécessaire. Ce type de passage nécessite d’utiliser la notion des pointeurs [5] [9].

Remarque :

Une illustration du passage de paramètre par valeur est donnée dans l’exemple 3 de la section

4. Le passage de paramètre par adresse sera illustré dans le chapitre suivant.

3. Les fonctions

Une fonction est un sous-programme qui admet un nom, un type et des paramètres. Une fonction admet un type et retourne toujours un résultat.

3.1. Définition d'une fonction

On définit une fonction comme suit :

Fonction *nom_fonction* (*paramètres : types*) : *type de la valeur retournée*

Var *var1, var2, ... : types*; // *variables locales*

Début

instructions de la fonction ;

Retourner (*résultat*) ; // (*au moins une fois*)

Fin Où :

- Les paramètres sont en nombre fixe ($n \geq 0$)
- Le type de la valeur retournée est le type de la fonction.
- La valeur de retour (ou le résultat) est spécifiée par l'instruction **Retourner**.

3.2. Appel d'une fonction

On fait appel à une fonction comme suit : *var* ←

nom_fonction (*paramètres*) ;

Où les paramètres d'appel peuvent être des variables, des constantes ou même des résultats d'une autre fonction. **Remarque :**

A la définition ou à l'appel d'une fonction, les parenthèses sont toujours présentes même lorsqu'il n'y a pas de paramètres.

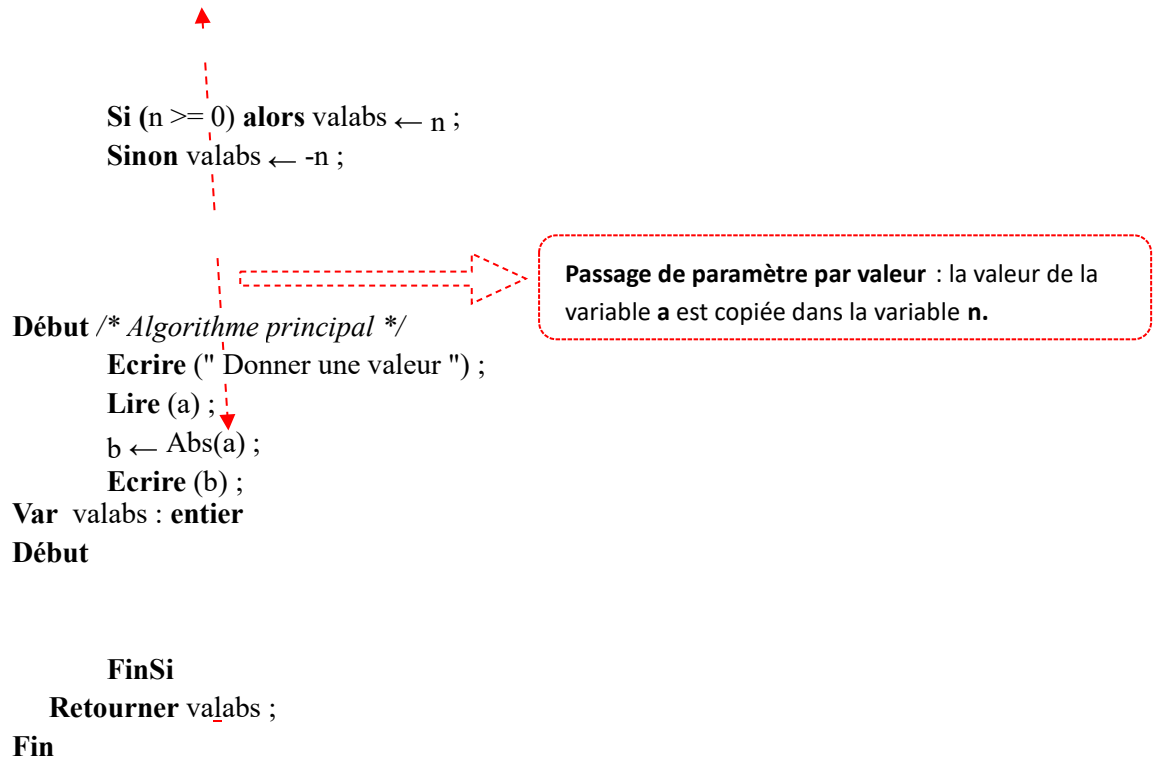
Exemple 1 :

Soit l'algorithme A qui calcule la valeur absolue d'un entier en utilisant une fonction :

Algorithme A ;

Var *a, b : entier* ;

Fonction Abs (*n : entier*) : *entier* /* Définition de la fonction */



Fin

Lors de l'exécution de la fonction `Abs()`, il y a une association entre le paramètre effectif **a** et le paramètre formel **n** d'où la valeur de **a** est copiée dans **n**. Ce type d'association s'appelle **passage de paramètre par valeur**.

4. Les procédures

Une procédure est un sous-programme qui admet également un nom et des paramètres mais ne retournant aucun résultat.

4.1. Définition d'une procédure

On définit une procédure comme suit :

```

Procédure nom_procedure (paramètres : types)
Var var1, var2, ... : types; // variables_locales
Début
    instructions de la procédure ; Fin
    
```

4.2. Appel d'une procédure

L'appel d'une procédure se fait comme suit : *nom_procedure*
(*paramètres*) ;

Exemple 2 :

Soit l'algorithme B calculant la valeur absolue d'un entier mais cette fois-ci en utilisant une procédure :

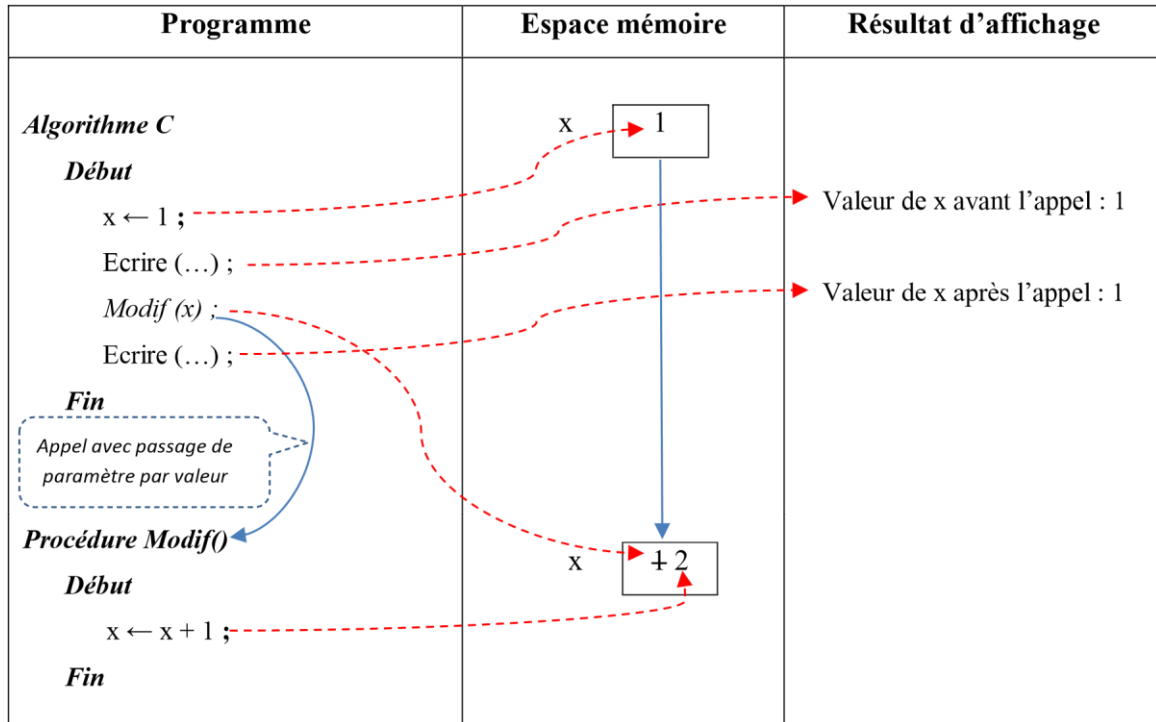
```
Algorithme B ;  
Var a : entier ;  
Procédure Abs (n : entier) /* Définition de la procédure */  
  Début  
    Si (n >= 0) alors  
      Ecrire n ;  
    Sinon  
      Ecrire -n ;  
    FinSi  
  Fin  
  
Début /* Algorithme principal */  
  Ecrire (" Donner une valeur ") ;  
  Lire (a) ;  
  Abs(a) ;  
  
Fin
```

Exemple 3 :

Soit un algorithme utilisant une procédure comme suit :

```
Algorithme C ;  
Var x : entier ;  
Procédure Modif (x : entier)  
  Début  
x ← x + 1 ; /* le x local est modifié, pas le x du programme principal */  
  Fin  
  
Début /* Algorithme principal */ x  
  ← 1 ;  
  Ecrire ("Valeur de x avant l'appel :", x) ;  
  Modif (x) ;  
  Ecrire ("Valeur de x après l'appel :", x) ; Fin
```

L'exécution de l'algorithme C peut être illustrée comme suit :



Lorsqu'on passe un paramètre à une fonction (ou à une procédure), cette dernière ne peut pas modifier la variable. La variable est automatiquement copiée et la fonction travaille sur une copie de la variable. La modification de la copie n'entraîne pas une modification de la variable originale. C'est ce qu'on appelle le passage de paramètre par valeur [3].

5. Fonctions et procédures récursives

La récursivité est une méthode de description d'algorithmes qui permet à une fonction (ou procédure) de s'appeler elle-même directement ou indirectement.

5.1. Exemple illustratif

On peut définir la factorielle d'un nombre N non négatif de deux manières :

Définition non récursive : $N! = N * N-1 * \dots * 2 * 1$

Définition récursive : $N! = N * (N - 1)!$ et $0! = 1$

Par conséquent, deux solutions sont possibles pour le calcul de la factorielle.

a) Solution itérative :

Fonction FACT (n : entier) : entier

Var i, F: entier ;

Début

Si (n = 0) **alors** F $\leftarrow$ 1 ;

Sinon

$F \leftarrow 1$;

Pour i de 2 à n

$F \leftarrow F * i$;

Finpour

Retourner F;

Finsi

Fin

b) Solution récursive :

Fonction FACT (n : entier) : entier

Début

Si (n=0) **alors** **Retourner** 1 ;

Sinon **Retourner** n * fact (n-1) ; // *Appel récursif de la fonction FACT*

Finsi **Fin**

5.2. Interprétation

Une procédure ou une fonction récursive doit comporter une condition d'arrêt (n=0 dans l'exemple étudié ci-dessus). Cette condition empêche des appels récursifs sans arrêt.

Généralement, la condition d'arrêt se présente sous la forme d'une instruction « Si...

Alors...Sinon » qui permet de stopper la récurrence si la condition d'arrêt est satisfaite.

Dans le cas contraire, la fonction ou la procédure continue à exécuter les appels récursifs.

D'autre part, le paramètre de l'appel récursif doit converger toujours vers la condition d'arrêt. Un processus récursif remplace en quelque sorte une boucle, ainsi tout processus récursif peut être également formulé en tant qu'un processus itératif.

5.3. Mécanisme de fonctionnement

Considérons, à titre d'exemple, le calcul de la factorielle de 4 en appliquant la fonction récursive FACT(). Pour calculer FACT(4), il faut calculer FACT(3). Pour calculer FACT(3), il faut calculer FACT(2). Pour calculer FACT(2), il faut calculer FACT(1). Pour calculer FACT(1), il faut connaître FACT(0), ce dernier vaut 1. Ensuite, on revient à rebours pour terminer le calcul pour 1, 2, 3 puis 4.

Il y a donc autant d'occurrences de paramètre n et d'appel récursif que de niveaux de récursivité. A chaque niveau, un nouvel environnement, comprenant les paramètres et les variables locales de la fonction, est créé. Cette gestion des variables est invisible à

l'utilisateur et effectuée automatiquement par le système si le langage admet la récursivité [5]. Une illustration de ce mécanisme est donnée dans la figure 6.

FACT(4) = 24

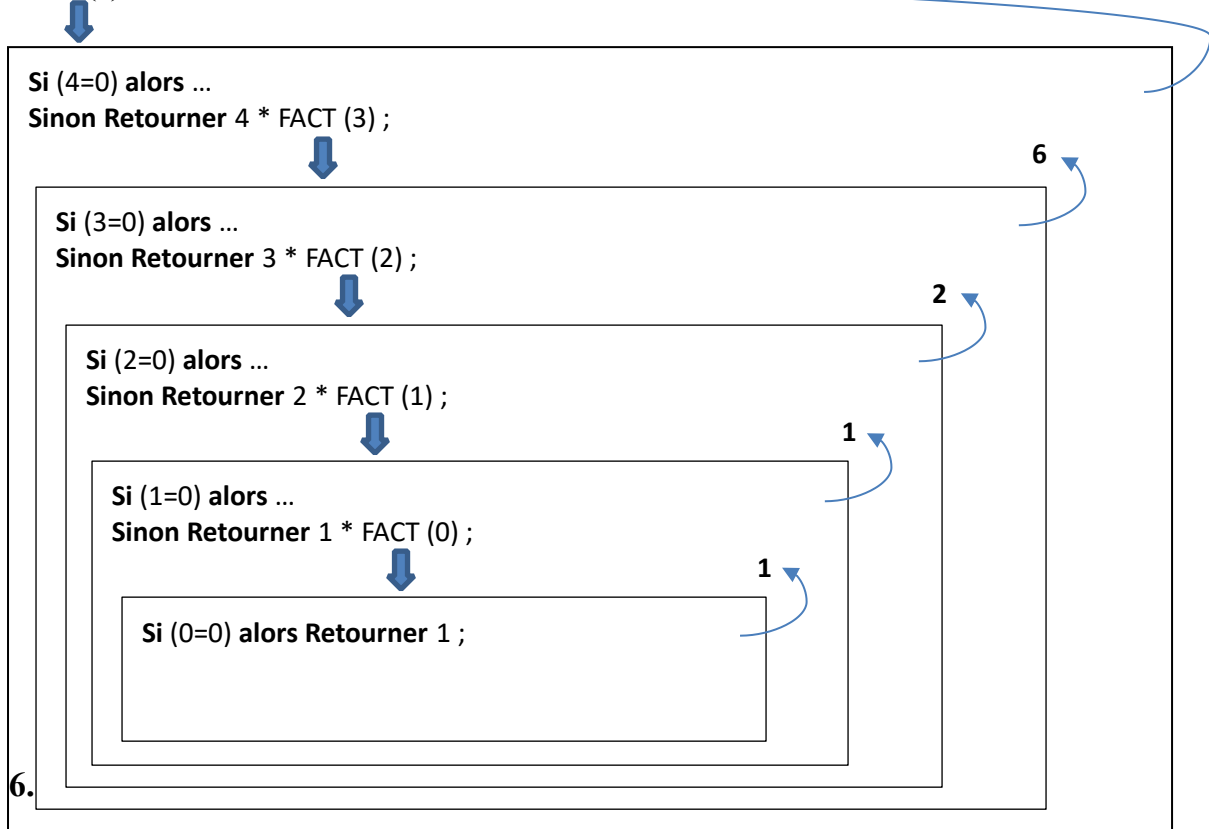


Figure 7. Calcul de la factorielle par récursivité.

6. Conclusion

La notion du sous-programme est très importante en programmation. Elle résulte de la décomposition du programme initial en de plus petites unités ou parties réutilisables qui sont ensuite appelées le moment opportun par le programme principal. Un sous-programme évite la répétition inutile de code et permet de clarifier le programme. En algorithmique, un sousprogramme correspond à une fonction ou à une procédure dont la description et le mécanisme de fonctionnement sont présentés plus haut dans ce chapitre. Aussi, une initiation à la notion de récursivité, une des outils puissants de la programmation, a été donnée à la fin de ce chapitre.

Le prochain chapitre traite l'un des concepts utiles et indispensables au passage de paramètres par adresse (ou référence), c'est le concept de pointeur.

Partie II - Exercices corrigés

Série 1 : Initiation aux algorithmes

Exercice 1

Soit l'algorithme suivant :

```
Algorithme A ;  
Var a, b, c : entier ;  
Début a ← 5 ;  
      b ← a+1 ;  
      c ← a+b ;  
Fin
```

- a) Expliquer chaque ligne, mot et symbole dans cet algorithme.
- b) Sur la base de cette explication, ajouter des commentaires à cet algorithme.
- c) A quoi servent ces commentaires ? Est-ce qu'ils sont obligatoires ?
- d) Dérouler l'algorithme et donner les valeurs des variables **a**, **b** et **c**.

Exercice 2

Soit l'algorithme suivant :

```
Algorithme B ;  
Var a, b, c, d : entier ;  
Début  
      Ecrire ("Donner a et b")  
      ; Lire (a, b) ; c ← a+b ;  
      d ← a*b ;  
      Ecrire (c, d) ;  
Fin
```

- a) Que fait cet algorithme ?
- b) Dérouler l'algorithme pour a=5 et b=6.
- c) Même question pour a=6 et b=5.

Exercice 3

Soit l'algorithme suivant :

```
Algorithme C ;
```

```
Var x,y :entier ;  
Début  
    Ecrire (Donner x, y)  
    ; Lire (x,y) ; x  $\square$   
    y+1 ; z  $\square$  z+x ;  
    Ecrire ('x,z') ;  
Fin
```

a) Corriger l'algorithme s'il y a des erreurs.

b) Dérouler l'algorithme pour x=9 et y=3.

Exercice 4

Soit l'algorithme suivant :

```
Algorithme D ;  
Var a, b, c : entier ;  
Début  
    Ecrire (donner a et b) ;  
    Lire ('a, b') ;  
    c  $\square$  a*a ;  
    d  $\square$  b*b ;  
    Ecrire (c, d) ;  
Fin
```

a) L'algorithme contient-il des erreurs ?

b) Dérouler l'algorithme pour a=5 et b=7 et déduire qu'est-ce qu'il fait.

Exercice 5

Corriger l'algorithme suivant, s'il le faut, et donner son résultat :

```
Algorithme A ;  
Var x, y, z, s : entier ;  
Début x  $\leftarrow$  10 ;  
    y  $\leftarrow$  15 ;  
    z  $\leftarrow$  20 ;  
    m  $\leftarrow$  (x + y + z) / 2 ;  
    Ecrire (m) ;  
    Ecrire (x + y + z / 2) ;
```

Fin

Exercice 6

- a) Quel résultat produira-t-il le déroulement de l'algorithme suivant ?

Algorithme B ;

Var val, double, triple : entier ;

Début val $\leftarrow$ 1000 ;

 double $\leftarrow$ val * 2 ;

 triple $\leftarrow$ val * 3 ;

 Ecrire val ;

 Ecrire double ;

 Ecrire triple ;

Fin

- b) Proposer une simplification de cet algorithme en produisant le même résultat.

Exercice 7

Supposons que l'on veut afficher à l'écran de l'utilisateur le message « Bonjour à tous ».

- a) Essayer d'écrire l'algorithme correspondant.
- b) Quelle est la sortie (output) de cet algorithme ? et l'entrée (input) ?
- c) Quelles sont les instructions à utiliser pour manipuler les entrées (et les sorties) d'un algorithme ?
- d) Reformuler votre algorithme pour qu'il y ait une entrée et une sortie.

Série 2 : Instructions algorithmiques de base

Exercice 1

- a. Ecrire un algorithme qui permet de lire un nombre (donné par l'utilisateur), puis il calcule et affiche son carré.
- b. Même question pour calculer et afficher le cube, ensuite l'inverse de ce nombre.

Exercice 2

- a. Ecrire un algorithme qui permet de lire les notes de trois matières ensuite il calcule et affiche leur moyenne.

- b. Modifier l'algorithme dans le cas où les matières ont des coefficients qui doivent être donnés avec les notes.

Exercice 3

Ecrire un algorithme qui permet de lire deux variables numériques a et b et de les afficher avant et après leur permutation.

Par exemple, avant : a=5 et b=7, après : a=7 et b=5.

Exercice 4

Proposer un algorithme qui réalise la permutation de deux variables numériques sans avoir utiliser une troisième variable.

Série 3 : Les instructions conditionnelles

Exercice 1

Ecrire un algorithme qui permet d'afficher la valeur absolue d'un nombre donné.

Exercice 2

Ecrire un algorithme qui permet de déterminer si un entier donné est pair ou impair.

Exercice 3

Ecrire un algorithme qui demande trois lettres à l'utilisateur, et l'informe ensuite si leur ordre de lecture et le même que l'ordre alphabétique.

Exercice 4

- a) Écrire un algorithme qui lit trois variables au clavier et affiche le maximum des trois.
- b) Même question pour plus de trois variables.

Exercice 5

- a) Ecrire un algorithme qui demande deux nombres à l'utilisateur et l'informe ensuite si leur produit est négatif ou positif mais sans le calculer. (On laisse de côté le cas où le produit est nul).
- b) Même question en incluant cette fois-ci le cas où le produit peut être nul.

Exercice 6

Ecrire un algorithme qui permet de lire un numéro du jour de la semaine (numéro entre 1 et

7) et d'afficher le nom du jour correspondant. Par exemple, le dimanche correspond au numéro 1.

Série 4 : Les instructions itératives

Exercice 1

Donner les affichages produits par l'exécution des algorithmes suivants :

Algorithme 1 :

Var i : entier ; Début

Pour i ← 2 à 8

Écrire ("Bonjour") ;

Écrire (i) ;

Fin pour

Écrire ("fin") ;

Fin

Algorithme 2 :

Var encore : booléen ; Début

encore ← Vrai ;

Répéter

Écrire ("Bonjour") ;

Jusqu'à (encore)

Écrire ("fin") ;

Fin

Algorithme 3 :

Var encore : booléen ;

Début encore ←

Faux ;

Tant que (encore) faire

Écrire ("Salut") ;

Fin tant que

Écrire ("fin") ;

Fin

Exercice 2

Ecrire l'algorithme qui affiche la somme des prix d'une suite d'articles saisie par l'utilisateur et se terminant par zéro (Justifier le choix de la boucle à utiliser).

Exercice 3

- a) Ecrire l'algorithme qui demande un entier, ensuite il affiche les dix entiers suivants. Par exemple, si l'on entre le nombre 10, l'algorithme affichera les nombres 11, 12,..., 20, 21.
- b) Modifier l'algorithme pour qu'il affiche les dix nombres pairs suivants.

Exercice 4

- a) Ecrire l'algorithme qui demande un nombre entier n , ensuite il affiche la somme des entiers positifs jusqu'à n .
Par exemple, si $n=5$, l'algorithme affiche : $1 + 2 + 3 + 4 + 5 = 15$
- b) Réécrire le même algorithme mais cette fois ci pour le calcul d'un produit. Par exemple, pour $n=5$, l'algorithme affiche : $1 \times 2 \times 3 \times 4 \times 5 = 120$

Remarque. On souhaite afficher uniquement le résultat sans la décomposition du calcul.

Exercice 5

Ecrire un algorithme qui permet de calculer la factorielle d'un entier (qui doit être positif).
Où $n! = n \times (n-1) \times (n-2) \times \dots \times 1$, si $n \geq 1$ et $n! = 1$ si $n = 0$.

Exercice 6

- a) Ecrire un algorithme qui demande successivement dix nombres à l'utilisateur, ensuite il affiche le plus petit parmi eux (le minimum).

Exemple :

Entrer le nombre numéro 1 : 12

Entrer le nombre numéro 2 : 3

... (on suppose que les autres nombres sont ≥ 3) Entrer le nombre numéro 10 : 6

Résultat :

Le minimum est : 3

- b) Modifier l'algorithme pour qu'il affiche, en plus, la position de ce nombre.

Pour l'exemple précédent, le minimum se trouve à la position : 2

- c) Réécrire l'algorithme précédent dans le cas où l'on ne connaît pas à l'avance combien de nombres l'utilisateur souhaite saisir, mais la saisie des nombres s'arrête lorsque l'utilisateur entre un zéro.

Exercice 7

Ecrire un algorithme qui demande un nombre puis vérifier si ce nombre est premier ou non

Exercice 8

- a) Supposons que le code pin d'un utilisateur est 5454. Ecrire un algorithme qui demande à cet utilisateur de saisir son code pin jusqu'à ce que la réponse convienne.
- b) Ajouter une condition qui annule la saisie après trois tentatives erronées.

Série 5 : Les tableaux et les structures

Exercice 1

- a) Ecrire un algorithme qui permet de lire 10 valeurs données par l'utilisateur en les stockant dans un tableau, ensuite l'algorithme doit afficher seulement les valeurs impaires.
- b) Modifier l'algorithme (a) pour que le nombre de valeurs soit donné par l'utilisateur.
- c) Réécrire l'algorithme (b) en divisant cette fois-ci le tableau initial en deux tableaux, l'un contenant les valeurs paires et l'autre contenant les valeurs impaires, et en les affichant par la suite.

Exercice 2

- a) Soit un tableau de 10 éléments réels, écrire un algorithme qui permet de lire ce tableau et rechercher le maximum des éléments ainsi que sa position, ensuite l'afficher.
- b) Même question pour un tableau de 10 lignes et 5 colonnes.

Exercice 3

Ecrire un algorithme qui permet de rechercher une valeur numérique saisie par l'utilisateur dans une matrice de taille $N \times M$ (N et M sont données).

Exercice 4

Ecrire l'algorithme qui permet de compter le nombre d'occurrences d'un élément donné dans une matrice de taille $N \times M$.

Exercice 5

Définir une structure Rationnel permettant de coder un nombre rationnel, avec numérateur et dénominateur. Ecrire ensuite l'algorithme qui permet la saisie, l'affichage, la multiplication et l'addition de deux rationnels. Pour l'addition, afin de simplifier, on ne cherchera pas nécessairement le plus petit dénominateur commun.

Exercice 6

Soit une structure Compte qui code un compte bancaire défini par un numéro, nom et prénom et date d'ouverture. Donner la définition des structures nécessaires.

Ecrire ensuite l'algorithme qui permet de saisir un tableau de N comptes et de l'afficher.

Série 6 : Les fonctions et les procédures

Exercice 1

Supposons qu'on veut écrire un sous-programme MoySom qui prend en paramètres trois entiers a, b et c, et qui affiche leur somme et renvoie leur moyenne.

Quelle est la déclaration correspondante ?

- **Fonction** MoySom (a : entier, b: entier, c: entier) : entier ;
- **Fonction** MoySom (a : entier, b: entier, c: entier) : réel ;
- **Procédure** MoySom (a : entier, b: entier, c: entier, som: entier, moy: réel) ;
- **Fonction** MoySom (a : entier, b: entier, c: entier) : entier, réel ;

Justifier la réponse choisie.

Exercice 2

- a) Écrire deux fonctions Min et Max qui retournent le minimum et le maximum de deux nombres réels donnés comme paramètres.
- b) Écrire deux autres fonctions Min_4 et Max_4 utilisant les fonctions Min et Max pour retourner le minimum et le maximum de quatre nombres réels passés en paramètres.
- c) Incorporer ces fonctions dans un algorithme complet.

Remarque : On suppose que tous les nombres donnés sont différents.

Exercice 3

Écrire un algorithme qui permet de lire une liste de N étudiants ayant comme informations :

numéro, nom, prénoms et moyenne du bac. L'algorithme doit afficher cette liste triée par moyenne en ordre décroissant.

Remarque : utiliser une fonction ou une procédure pour le tri.

Exercice 4

Un magasin de vente des composants électroniques vend quatre types de produits :

- Des cartes mères (code 1) ;
- Des processeurs (code 2) ;
- Des barrettes de mémoire (code 3) ;

- Des cartes graphiques (code 4).

Chaque produit possède une référence (qui est un nombre entier), un prix en DA et une quantité disponible.

- a) Définir une structure Produit qui code un produit.
- b) Ecrire une fonction qui permet la saisie et l’affichage des données d’un produit.
- c) Ecrire une fonction qui permet à un utilisateur de saisir une commande d’un produit.

L’utilisateur saisit la quantité commandée et les données du produit. L’ordinateur affiche toutes les données de la commande, y compris le prix.

Exercice de réflexion (sans correction)

Exercice A :

On se propose de réaliser une calculatrice qui permet de faire les opérations arithmétiques de base (+, -, * et /) de deux nombres a et b , ainsi que de savoir :

- si la somme $a + b$ est paire ;
- si le produit $a*b$ est pair ;
- le signe de la somme $a + b$; - le signe du produit $a*b$.

Ecrire l'algorithme correspondant.

Exercice B :

On se propose de faire la rotation (décalage des positions) des éléments d'un tableau.

1. Proposer un algorithme pour ce problème sachant que le degré de rotation doit être défini par l'utilisateur. Exemple pour un degré de rotation égal à 2 : Avant rotation : 1,3,9,4,5,2 □ Après rotation : 5,2,1,3,9,4
2. Modifier l'algorithme pour que la rotation puisse être faite dans les deux sens (avantarrière).

Exercice C :

Une société désire organiser un concours de recrutement en deux spécialités S1 et S2.

Le concours se porte sur trois (03) matières : M1, M2 et M3.

Un candidat est retenu si les conditions suivantes sont toutes remplies :

- Il doit se classer parmi les dix (10) premiers,
- Il ne doit pas avoir eu un zéro dans l'une des trois matières,
- Sa spécialité doit correspondre à l'une des deux spécialités concernées.

La note de classement est la moyenne des notes des trois matières.

Ecrire l'algorithme qui permet de lire les informations nécessaires, de classer tous les candidats par ordre de mérite, ensuite afficher la liste des candidats retenus pour le recrutement.

Exercice D :

Une entreprise veut gérer la liste de ses employés en les stockant dans un tableau. Sachant que chaque employé possède comme information : un matricule, un nom et prénom, une date de naissance, une situation familiale et une adresse. Cette dernière est définie par un numéro, rue, ville et code postale.

1. Donner la fonction ou la procédure qui permet de :

- rechercher un employé par son matricule et afficher ses informations.
 - afficher les nom et prénoms des employés qui habitent une même ville donnée.
2. Ecrire l'algorithme qui permet de créer la liste des employés et de faire appel à ces fonctions (ou procédures).

Corrigé série 1

Solution 1 :

a) Algorithme A ;

```
Var a, b, c : entier ; // déclaration des variables
Début a ← 5 ; // affectation de la valeur 5 à la variable a
      b ← a+1 ; // affectation du résultat de a+1 à la variable b
      c ← a+b ; // affectation de la somme de a et b à la variable c
Fin
```

b) Les commentaires dans un algorithme (ou programme) sont ajoutés à titre indicatif pour le rendre plus lisible et compréhensible mais ils ne sont pas obligatoires. c) Le résultat du déroulement de cet algorithme est comme suit :

$$a=5 \quad b=5+1=6$$

$$c=6+5=11$$

2 :

- a) L'algorithme calcule la somme et le produit de deux entiers
- b) $C=5+6=11$
- c) $D=5 \times 6=30$

Solution 3 :

a) Algorithme C ;

```
Var x,y :entier ;
Const z =10 ;
Début
    Ecrire ("donner x, y") ;
    Lire (x,y) ; x ← y+1 ;
    z ← z+x ; (z est une constante)
    Ecrire ('x,z') ;
Fin
```

b) Déroulement de l'algorithme pour $x=9$ et $y=3$ $x=3+1=4$

Affichage : $x=4$, $z=10$

Solution 4 :

a)


```
Algorithme D ;  
Var a, b, c : entier ;  
Début  
    Ecrire ("donner a et b")  
    ; Lire (a, b) ; c ← a ; a ←  
    b ; b ← c ;  
    Ecrire (a, b) ;  
Fin
```

b) Déroulement de l'algorithme pour a=5 et b=7

c=5 a=7 b=5

Affichage : a=7 et b=5

Donc, l'algorithme réalise la permutation de a et b.

Solution 5 :

s : variable déclarée sans être utilisée

m : doit être de type réel Affichage :

22.5 ; 35

Solution 6 :

a) val=1000 ; double=2000 ; triple=3000

b) Simplification :

```
Var val : entier ;  
Début    val ←  
1000 ;  
    Ecrire (val, val*2, val*3) ;  
Fin
```

Solution 7 :

a) Algorithme Un_Bonjour ;

```
Début  
    Ecrire ("Bonjour à tous") ;  
Fin
```

Cet algorithme n'a qu'une sortie, le message « Bonjour à tous ».

b) Pour manipuler les entrées et les sorties d'un algorithme, on utilise respectivement les deux instructions : Lire() et Ecrire().

c) Algorithme Un_ReBonjour ;

Var mge : chaîne ;

Début

Ecrire ("Veuillez saisir un message :") ;

Lire (mge) ;

Ecrire (mge) ;

Fin

Corrigé série 2

Solution 1 :

a) Algorithme carré ;

Var nb, carr : réel ;

Début

Lire (nb) ;

$carr \leftarrow nb * nb$; // ou $carr \leftarrow nb^2$

Ecrire (carr) ;

Fin

b) Algorithme cube ;

Var nb, cub : réel ;

Début

Lire (nb) ;

$cub \leftarrow nb^3$

Ecrire (cub) ;

Fin

Algorithme inverse ;

Var nb, inv : réel ;

Début

Lire (nb) ; $inv \leftarrow 1 / nb$; //

tel que $nb \neq 0$

Ecrire (inv) ; Fin

Solution 2 :

a. Algorithme moyenneA ;

Var n1, n2, n3, moy : réel ;

Début

Ecrire ("Donner trois notes :") ; // *ce message est optionnel*

Lire (n1, n2, n3) ; $moy \leftarrow (n1+n2+n3) / 3$;

Ecrire (moy) ;

Fin

b. Algorithme moyenneB ;

Var n1, n2, n3, moy : réel ; c1, c2, c3 : entier ;

Début

```

Lire (n1, n2, n3) ;
Lire (c1, c2, c3) ;
moy ← (n1*c1+n2*c2+n3*c3) / (c1+c2+c3) ;
Ecrire (moy) ;
Fin

```

Solution 3 :

```

Algorithme permutation ;
Var a,b,c : réel ;
Début
    Ecrire ("Entrer la valeur de a :") ;
    Lire (a) ;
    Ecrire ("Entrer la valeur de b :") ; Lire
    (b) ;
    Ecrire ("a=",a) ;
    Ecrire ("b=",b) ;
    c ← a ;

```

La permutation est réalisée à travers ces trois affectations.

a ← b ; La variable « c » est une variable temporaire qui doit être de même type que a et b.
b ← c ;

```

    Ecrire ("a=",a) ;
    Ecrire ("b=",b) ;
Fin

```

Solution 4 :

```

Algorithme permutation2 ;
Var x, y : réel ;
Début
    Lire (x,y) ;
    x ← x + y ;
    y ← x - y ;
    x ← x - y ;
    Ecrire (x,y) ;
Fin

```

Corrigé série 3

Solution 1 : (L'écriture des entêtes d'algorithmes est omise dans quelques solutions) Var

n : réel ;

Début

Ecrire ("Entrez un nombre : ") ;

Lire (n) ;

Si ($n < 0$) Alors $n \leftarrow -n$;

FinSi

Ecrire ("la valeur absolue est", n) ; Fin

Solution 2 :

Var n : entier ;

Début

Ecrire ("Entrez un nombre : ") ;

Lire (n) ;

Si ($n \bmod 2 = 0$) Alors Ecrire ("Ce nombre est pair") ;

Sinon Ecrire ("Ce nombre est impair") ;

FinSi

Fin

Solution 3 :

Var a, b, c : caractère ;

Début

Ecrire ("Entrez successivement trois lettres : ") ;

Lire (a, b, c) ;

Si ($a < b$ ET $b < c$) Alors Ecrire ("Les lettres sont classées alphabétiquement") ;

Sinon Ecrire ("Les lettres ne sont pas classées") ;

FinSi Fin

Solution 4 :

a) Var a, b, c, max ;

Début

Ecrire ("Entrez trois nombres réels : ") ;

Lire (a, b, c) ;

Si ($a < b$) Alors $\max \leftarrow b$;

Sinon $\max \leftarrow a$;

FinSi

Si ($\text{max} < c$) Alors $\text{max} \leftarrow c$; FinSi

Ecrire ("Le maximum des trois nombres est :", max);

Fin Autre variante :

...

Lire (a, b, c);

$\text{max} \leftarrow a$;

Si ($\text{max} < b$) Alors $\text{max} \leftarrow b$; FinSi

Si ($\text{max} < c$) Alors $\text{max} \leftarrow c$; FinSi

Ecrire ("Le maximum des trois nombres est :", max);

Fin

b) Même principe pour 4 et 5 variables (juste pour illustrer l'utilité de la variable max).

Solution 5 : a)

Var m, n : entier ;

Début

Ecrire ("Entrez deux nombres : ");

Lire (m, n);

Si ($(m > 0 \text{ ET } n > 0) \text{ OU } (m < 0 \text{ ET } n < 0)$) Alors Ecrire ("Le produit est positif");

Sinon Ecrire ("Le produit est négatif");

FinSi

Fin

b)

Var m, n : entier ;

Début

Ecrire ("Entrez deux nombres : "); Lire (m, n);

Si ($m = 0 \text{ OU } n = 0$) Alors Ecrire ("Le produit est nul");

Sinon Si ($(m < 0 \text{ ET } n < 0) \text{ OU } (m > 0 \text{ ET } n > 0)$) Alors

Ecrire ("Le produit est positif");

Sinon Ecrire ("Le produit est négatif");

FinSi

FinSi

Fin

Solution 6 :

Var j : entier ;

Début

Ecrire ("Donner le numéro du jour :"); Lire (j);

Selon j

1 : Ecrire ("Dimanche");

2 : Ecrire ("Lundi");

3 : Ecrire ("Mardi");

4 : Ecrire ("Mercredi");

5 : Ecrire ("Jeudi");

6 : Ecrire ("Vendredi");

7 : Ecrire ("Samedi");

Défaut : Ecrire ("Donner un numéro de jour valide (entre 1 et 7).");

FinSelon

Fin

Corrigé série 4

Solution 1 :

Algorithme 1 :

```
Bonjour
2
Bonjour
3
Bonjour
4
Bonjour
5
Bonjour
6
Bonjour
7
Bonjour
8
Fin
```

Algorithme 2 :

```
Bonjour
Fin
```

Algorithme 3 :

```
Fin
```

Solution 2 :

```
Var p, s : réel ;
Début s ← 0 ;
  Répéter
    Ecrire ("Entrer le prix de l'article (0 si
    fin):"); Lire (p) ; s ← s + p ;
  Jusqu'à (p = 0)
  Ecrire (" La somme des prix des articles est ", s) ;
Fin
```

Puisque le nombre d'articles n'est pas connu à l'avance ainsi qu'il faut faire au moins une saisie pour terminer, la boucle « Répéter » est appliquée dans ce cas.

Solution 3 :

a) Var N, i : Entier ;

Début

Ecrire ("Entrer un entier : ") ; Lire (N) ;

Ecrire ("Les 10 nombres suivants sont : ")

Pour i de (N + 1) à (N + 10)

Ecrire (i) ; /* Si (i mod 2 = 0) Ecrire i ; */

FinPour

Fin

b) Pour le deuxième cas, il suffit de remplacer l'instruction : « Ecrire (i) » par « Si (i mod 2 = 0) Alors Ecrire i » dans la 6^{ème} ligne.

Solution 4 : (les instructions correspondantes à la question b sont mises en commentaires)

Var N, i, som : Entier ;

Début

Ecrire ("Donner un entier : ") ;

Lire (N) ; som $\leftarrow$ 0 ; // prod $\leftarrow$ 1 ;

Pour i de 1 à N

som $\leftarrow$ som + i ; // prod $\leftarrow$ prod * i ;

FinPour

Ecrire ("La somme = ", som) ; // Ecrire ("Le produit = ", prod) ; Fin

Solution 5 :

Var N, i, F : entier ;

Début

Ecrire ("Entrer un entier positif : ") ; Lire (N) ;

Si (N < 0) Alors Ecrire ("Erreur !") ;

Sinon

F $\leftarrow$ 1 ;

Si (N > 1) Alors

Pour i de 2 à N

F $\leftarrow$ F * i ;

FinPour

Fsi

Ecrire ("La factorielle est ", F) ;

Fsi

Fin

Autre possibilité :

Sinon

F $\leftarrow$ N ;

Tant que (N > 1) faire

N $\leftarrow$ N - 1 ;

F $\leftarrow$ F * N ;

FinTq

Fsi

Solution 6 :**a) et b)**

```
Var N, i, min, pmin : Entier ; Début
  min ← 0 ; // juste pour initialiser la var
  Pour i de 1 à 10
    Ecrire ("Entrer le nombre numéro ", i) ; Lire (N) ;
    Si (i = 1 ou N < min) Alors min ← N ; pmin ← i ;
  FinSi
FinPour
Ecrire ("Le minimum est : ", min) ;
Ecrire ("Il a été saisi en position :", pmin) ;
Fin
```

c/

```
Var N, i, min, pmin : Entier ; Début
  i ← 1 ; min ← 0 ;
  Répéter
    Ecrire ("Entrer le nombre numéro ", i) ; Lire (N) ;
  Si (i = 1 ou N < min) Alors min ← N ; pmin ← i ;
  FinSi i ← i + 1 ;
  Jusqu'à (N=0)
  Ecrire ("Le minimum est ", min),
  Ecrire ("Il a été saisi en position numéro ", pmin),
Fin
```

Solution 7 :

Algorithme nombre_premier

Var i, N : entier ;

x : booleen;

Début

```
Ecrire ("entrer N"); Lire(N);
x ← faux;
i ← 2;
Tant que (i < N et x = faux) Faire
  Si (N mod i = 0) alors
    Ecrire ("le nombre n'est pas premier") ;
```

```
        x ← vrai ;
        i ← i+1;
    Finsi
Fin Tq
Si (x=faux) alors Ecrire ("le nombre est premier") ;
Fin
```

Solution 8 :

a) et b)

```
Algorithme codePin ;
Var codePin, monCode, tentative: Entier ; Début
    codePin ← 5454 ;
    tentative ← 3 ;
    Répéter
        Ecrire ("Entrez le code PIN :") ; Lire (monCode) ;
        tentative ← tentative – 1 ;
        Si (monCode ≠ codePin) Alors Ecrire ("Code incorrect") ; FinSi
    Jusqu'à (monCode = codePin ou tentative=0)
    Si (tentative=0) Alors
        Ecrire ("Vous ne pouvez plus saisir de code") ;
    Sinon
        Ecrire ("Bienvenue") ;
    FinSi
Fin
```

Corrigé série 5

Solution 1 :

```
Algorithme a ;
Var i : entier ;
Tableau T[10] : entier ;
Début
    Pour i de 0 à 9
        Ecrire ("Donner un entier : ") ; Lire T[i] ;
    FinPour
    Pour i de 0 à 9
        Si (T[i] mod 2 <> 0) Alors Ecrire T[i] ;
```

FinPour

Fin

Algorithme b ;

Var i, n: entier ;

Tableau T[n] : entier ;

Début

Ecrire ("Donner le nombre de valeurs à saisir ") ; Lire (n) ;

Pour i de 0 à n-1

Ecrire ("Donner un entier : ") ; Lire T[i] ;

FinPour

Pour i de 0 à n-1

Si $T[i] \bmod 2 \neq 0$ Alors Ecrire T[i] ; FinPour

Fin

Algorithme_c ;

Var i, n, j, k: entier ;

Tableau T[n], Ti[n], Tp[n] : entier ; // on suppose par défaut que les trois tableaux ont la même taille

Début

Ecrire ("Donner le nombre de valeurs à saisir ") ; Lire (n) ;

Pour i de 0 à n-1

Ecrire ("Donner un entier : ") ; Lire T[i] ;

FinPour

$j \leftarrow 0$; $k \leftarrow 0$;

Pour i de 0 à n-1

Si $(T[i] \bmod 2 \neq 0)$ Alors $Ti[j] \leftarrow T[i]$; $j \leftarrow j+1$;

Sinon $Tp[k] \leftarrow T[i]$; $k \leftarrow k+1$;

FinPour

Pour i de 0 à j-1

Ecrire Ti[i] ;

FinPour

Pour i de 0 à k-1

Ecrire Tp[i] ;

FinPour

Fin

Solution 2:

a)

```
Algorithme Max_tableau1D
Var i, pos: entier ; max : réel ;
Tableau A [10]: réel ;
Début
    Pour i de 0 à 9
        Ecrire ("Entrer un nombre:"); Lire (A[i]) ;
        Si (i = 0 OU max < A[i]) Alors
            max ← A[i]; pos ← i ;
        Finsi
    Finpour
    Ecrire ("Le maximum du tableau est :", max);
    Ecrire ("Il se trouve à la position", pos+1);
Fin
```

b)

```
Algorithme Max_tableau2D
Var i,j, pos: entier ; max : réel ;
Tableau A[5] [10]: réel ;
Début
    Pour i de 0 à 4
        Pour j de 0 à 9
            Ecrire ("Entrer un nombre:");
Lire (A[i][j]) ;
            Si (i = 0 ET j = 0 OU max < A[i] [j]) Alors
                max ← A[i] [j] ; lin ← i ; col ← j ;
            Finsi
        Finpour
    Finpour
    Ecrire ("Le maximum du tableau est :", max);
    Ecrire ("Il est positionné à la ligne", lin+1, "et la colonne", col+1); Fin
```

Solution 3 :

```
Algorithme Recherche_tableau2D
Var i, j, n, m: entier ; v : réel ; drap : booléen ;
Tableau A[n][m]: réel ;
Début
```

```
Ecrire ("Donner le nombre de lignes:"); Lire(n) ;
Ecrire ("Donner le nombre de colonnes:"); Lire(m) ;
Pour i de 0 à n-1
    Pour j de 0 à m-1
        Ecrire ("Entrer un nombre:"); Lire (A[i][j]) ;
    Finpour
Finpour
Ecrire ("Donner la valeur recherchée:"); Lire(v) ;
drap ← Faux ; i ← 0 ; j ← 0 ; Tant que (i < n ET
drap=Faux) faire
Tant que (j < m ET drap=Faux) faire
Si (A[i][j]=v) Alors drap ← Vrai ;
    Finsi
FinTantque
FinTantque
Si (drap=Vrai) Alors
    Ecrire ("La valeur recherchée existe dans le tableau");
Sinon
    Ecrire ("La valeur recherchée n'existe pas dans le tableau");
Finsi
Fin
```

Solution 4 :

```
Algorithme Occurence_tableau2D
Var i, j, n, m, compt: entier ; v : réel ;
Tableau A[n][m]: réel ;
Début
    Ecrire ("Donner le nombre de lignes:"); Lire(n) ;
    Ecrire ("Donner le nombre de colonnes:"); Lire(m) ;
    Pour i de 0 à n-1
        Pour j de 0 à m-1
            Ecrire ("Entrer un nombre:");
            Lire (A[i][j]) ;
        Finpour
    Finpour
```

```
Ecrire ("Donner une valeur:"); Lire(v) ; compt←
0 ; i←0 ; j←0 ;
Pour i de 0 à n-1
    Pour j de 0 à m-1
        Si (A[i][j]=v) Alors compt← compt+1 ;
    Finsi
Finpour
Finpour
Ecrire ("Le nombre d'occurrences de la valeur donnée est :", compt);
Fin
```

Solution 5 :

```
Algorithme Calcul_rationnel ;
Structure Rationnel // Définition de la structure d'un nombre rationnel
    numérateur: entier;
    dénominateur: entier;
FinStructure ;
Var  p, q, r : Structure Rationnel ;
Début
    /* Saisie */
    Ecrire ("Entrez le numérateur et le dénominateur du premier nombre : ");
    Lire (p.numérateur, p.dénominateur);

    Ecrire ("Entrez le numérateur et le dénominateur du deuxième nombre : ");
    Lire (q.numérateur, q.dénominateur);
    /* Affichage */
    Ecrire (p.numérateur, p.dénominateur);
    Ecrire (q.numérateur, q.dénominateur);
    /* Multiplication*/
    r.numérateur ← p.numérateur * q.numérateur ;
    r.dénominateur ← p.dénominateur * q.dénominateur ;
    Ecrire (r.numérateur, r.dénominateur);

    /* Addition*/
    r.numérateur ← p.numérateur * q.dénominateur + q.numérateur * p.dénominateur;
    r.dénominateur ← p.dénominateur * q.dénominateur; Ecrire
    (r.numérateur, r.dénominateur);
Fin
```

Solution 6 :

```
Algorithme Tableau_comptes ;
Structure Date jour, mois, annee :
    entier ;
FinStructure ;
Structure Compte num:
    entier; nom, pnom:
    chaine ; dtOuvert :
    Date ;
FinStructure ;
Var i , n : entier ;
Tableau C[n] : Structure Compte ;
Début
    Ecrire ("Nombre des comptes : ") ;
    Lire (n) ;
    Pour i de 0 à n-1
        Ecrire ("No du compte: ") ;
        Lire(C[i].num) ;
        Ecrire ("Nom et prénom du client : ") ;
        Lire(C[i].nom, C[i].pnom) ;
        Ecrire ("Date de naissance du client : ") ;
        Lire(C[i].dtOuvert.jour, C[i].dtOuvert.mois, C[i].dtOuvert.annee) ;
    Finpour
    Pour i de 0 à n-1
        Ecrire ("N°:" C[i].num, "Nom et prénom:", C[i].nom,
            "Date de naissance:", C[i].dtOuvert.jour, C[i].dtOuvert.mois,
            C[i].dtOuvert.annee) ;
    Finpour
Fin
```


Corrigé série 6

Solution 1 :

La déclaration qui correspond à ce sous-programme est : b

Fonction MoySom (a : **entier**, b: **entier**, c: **entier**) : **réel** ;

Cette fonction prend trois paramètres entiers et elle est de type réel puisqu'elle va envoyer une valeur réelle (la moyenne).

Solution 2 :

a.

Fonction Min (n1, n2 : réel) : réel

Début

Si (n1 < n2) Alors Retourner n1 ;

Sinon Retourner n2 ;

FinSi

Fin

Fonction Max (n1, n2 : réel) : réel

Début

Si (n1 < n2) Alors Retourner n2 ;

Sinon Retourner n1 ;

FinSi

Fin

b.

Fonction Min_4 (n1, n2, n3, n4 : réel) : réel

Début

Si (Min (n1, n2) < Min (n3, n4)) Alors Retourner Min (n1, n2) ;

Sinon Min (n3, n4) ;

FinSi

Fin

Fonction Max_4 (n1, n2, n3, n4 : réel) : réel

Début

Si (Max (n1, n2) > Max (n3, n4)) Alors Retourner Max (n1, n2) ;

Sinon Max (n3, n4) ;

FinSi

Fin

c.

Algorithme Max_Min

Var a, b, c, d, min, max : réel;

... */* Ici on écrit les fonctions définies ci-dessus */*

Début */* Algorithme principal */*

Ecrire (" Donner deux valeurs : ") ; Lire (a,b) ;

min ← Min (a,b) ; *// Appel de la fonction Min*

max ← Max (a,b) ; *// Appel de la fonction Max*

Ecrire (" Le minimum et le maximum des deux valeurs données sont
respectivement: min, max") ;

/ Cas de quatre nombres */*

Ecrire (" Donner quatre valeurs : ") ; Lire

(a,b,c,d) ; min ← Min_4 (a,b) ; *// Appel de la*

fonction Min max ← Max_4 (a,b) ; *// Appel de la*
fonction Max

Ecrire (" Le minimum et le maximum des valeurs données sont respectivement:
min, max") ; Fin

Solution 3 :

Algorithme Tri_Liste ;

/ Déclaration de l'enregistrement Etudiant */*

Structure Etudiant num:

entier; nom, pnom:

chaîne ; moy : réel ;

FinStructure ;

/ Déclaration de la liste des étudiants et des autres variables nécessaires */*

Var i , N : entier ;

Tableau Liste [N] : Structure Etudiant ;

*/*Déclaration du prototype de la procédure de tri, la définition est laissée à la fin*/*

Procédure Tri (Tableau T : Structure Etudiant, N : entier) ;

/ programme principal */*

Début

Ecrire ("Donner le nombre des étudiants : ") ;

Lire (N) ;

Pour i de 0 à N-1

Ecrire ("Numéro d'étudiant: ") ; Lire (Liste[i].num) ;

Ecrire ("Nom et prénom : ") ; Lire (Liste[i].nom, Liste[i].pnom) ;

Ecrire ("La moyenne du bac : ") ; Lire (Liste[i].moy) ;

Finpour

```
Tri(Liste,N) ; // Appel de la procédure Tri
Ecrire ("La liste des étudiants triés par moyenne est:") ;
Pour i de 0 à N-1
    Ecrire ("N:"Liste[i].num,"Nom et prénom: ", Liste[i].nom,
    Liste[i].pnom,"Moyenne:", Liste[i].moy) ;
Finpour
Fin

/* On applique dans ce qui suit l'algorithme de tri par sélection vu au cours selon un ordre
décroissant sur la moyenne */

Procédure Tri (Tableau T : Structure Etudiant, N : entier)
Var i, posmax : entier ; temp : Structure Etudiant ;
Début
    Pour i de 0 à N-2 posmax
        ← i ;
        Pour j de i + 1 à N-1
            Si (T(j) > T(posmax)) Alors posmax ← j ; Finsi
        Finpour
        Si (posmax ≠ i) Alors temp
            ← T(posmax) ;
            T(posmax) ← T(i) ;
            T(i) ← temp ;
        Finsi
    Finpour
Fin
```

Solution 4 :

```
Algorithme Gestion_vente ;
Structure Produit // Définition de la structure Produit
    ref, quantite : entier; type : chaine; prix : réel ;
FinStructure ;
Var p : Structure Produit ;
/* Définition des sous-programmes */
Fonction Saisie () : Structure Produit
Var p : Structure Produit ;
Début
    Ecrire ("Entrez le type du produit : ");
    Lire (p.type);
    Ecrire ("Entrez la référence : ");
```

```
Lire (p.ref);  
Ecrire ("Entrez le prix : ");  
Lire (p.prix);  
Ecrire ("Entrez la quantité : ");  
Lire (p.quantite);  
Retourner(p) ;
```

Fin

Procédure Affichage (p : Structure Produit)

Début

```
Ecrire ("Type :", p.type);  
Ecrire ("Référence : ", p.ref);  
Ecrire ("Prix : ", p.prix, "DA");  
Ecrire ("Quantité : ", p.quantite);
```

Fin

Procédure Commande_prod ()

Var qte : entier ; p :

Structure Produit ;

Début p ←

```
Saisie() ;  
Ecrire ("Entrez la quantité commandée : ");  
Lire (qte);  
Ecrire ("Récapitulatif de la commande :");  
Affichage(p);  
Ecrire ("Valeur de la commande : ", p.prix * qte, "DA");
```

Fin

Début */* Programme principal */*

Commande_prod () ;

Fin

Partie III

APPENDICITE

**LES BOUCLES, LES TABLEAUX ET LES ALGORITHMES
D'ORDONNANCEMENT ...**

*La véritable Université de nos jours, est une collection
des livres.*

ALGORITHMIQUE :EXERCICES SUR LES BOUCLES
TRAVAIL DE GROUPE
EXECUTEZ CHAQUE EXERCICE EN ALGOBOX, EN PYTHON, EN C OU EN JAVA ET EXPLIQUEZ
CHACUNE DES INSTRUCTIONS

EXERCICES SUR LES BOUCLES

TRAVAIL DE GROUPE

*EXECUTEZ CHAQUE EXERCICE EN ALGOBOX, EN PYTHON, EN C OU EN
JAVA ET EXPLIQUEZ CHACUNE DES INSTRUCTIONS*

Exercice A

Ecrire un algorithme qui demande à l'utilisateur un nombre compris entre 1 et 10 jusqu'à ce que la réponse convienne.

Exercice B

Ecrire un algorithme qui demande un nombre compris entre 10 et 50, jusqu'à ce que la réponse convienne. En cas de réponse supérieure à 50, on fera apparaître un message : « Plus petit ! », et inversement, « Plus grand ! » si le nombre est inférieur à 10.

Exercice C

Ecrire un algorithme qui demande un nombre de départ, et qui ensuite affiche les dix nombres suivants. Par exemple, si l'utilisateur saisit le nombre 18, le programme affichera les nombres de 19 à 28.

Exercice D

Réécrire l'algorithme précédent, en utilisant cette fois l'instruction **Pour**

Exercice E

Ecrire un algorithme qui demande un nombre de départ, et qui ensuite écrit la table de multiplication de ce nombre, présentée comme suit (cas où l'utilisateur entre le nombre 7) :
Table de 7 :

$$7 \times 1 = 7$$

$$7 \times 2 = 14$$

$$7 \times 3 = 21$$

...

$$7 \times 10 = 70$$

Exercice F

Ecrire un algorithme qui demande un nombre de départ, et qui calcule la somme des entiers jusqu'à ce nombre. Par exemple, si l'on entre 6, le programme doit calculer :

$$1 + 2 + 3 + 4 + 5 + 6 = 21$$

NB : on souhaite afficher uniquement le résultat, pas la décomposition du calcul.

Exercice G

Ecrire un algorithme qui demande un nombre de départ, et qui calcule sa factorielle.

NB : la factorielle de 5, notée 5 !, vaut

$$1 \times 2 \times 3 \times 4 \times 5$$

Exercice H

Ecrire un algorithme qui demande successivement 50 nombres à l'utilisateur, et qui lui dise ensuite quel était le plus grand parmi ces 50 nombres :

Entrez le nombre numéro 1 : 12

Entrez le nombre numéro 2 : 14 etc.

Entrez le nombre numéro 50 : 45

Le plus grand de ces nombres est : 45

Modifiez ensuite l'algorithme pour que le programme affiche de surcroît en quelle position avait été saisie ce nombre :

C'était le nombre numéro 50

Exercice I

Réécrire l'algorithme précédent, mais cette fois-ci on ne connaît pas d'avance combien l'utilisateur souhaite saisir de nombres. La saisie des nombres s'arrête lorsque l'utilisateur entre un zéro.

Exercice J

Lire la suite des prix (en franc congolais entiers et terminée par zéro) des achats d'un client. Calculer la somme qu'il doit, lire la somme qu'il paye, et simuler la remise de la monnaie en affichant les textes "10000 Francs" "5000 Francs" et "10000 francs" autant de fois qu'il y a de coupures de chaque sorte à rendre.

Exercice K

Écrire un algorithme qui permette de connaître ses chances de gagner au tiercé, quarté, quinté et autres impôts volontaires.

On demande à l'utilisateur le nombre de chevaux partants, et le nombre de chevaux joués. Les deux messages affichés devront être :

Dans l'ordre : une chance sur X de gagner

Dans le désordre : une chance sur Y de gagner

X et Y nous sont donnés par la formule suivante, si n est le nombre de chevaux partants et p le nombre de chevaux joués (on rappelle que le signe ! signifie "factorielle", comme dans l'exercice 5.7 ci-dessus) :

$$X = n ! / (n - p) !$$

$$Y = n ! / (p ! * (n - p) !)$$

ALGORITHMIQUE :EXERCICES SUR LES BOUCLES
TRAVAIL DE GROUPE
EXECUTEZ CHAQUE EXERCICE EN ALGOBOX, EN PYTHON, EN C OU EN JAVA ET EXPLIQUEZ
CHACUNE DES INSTRUCTIONS

NB : cet algorithme peut être écrit d'une manière simple, mais relativement peu performante. Ses performances peuvent être singulièrement augmentées par une petite astuce. Vous commencerez par écrire la manière la plus simple, puis vous identifierez le problème, et écrirez une deuxième version permettant de le résoudre.

RESOLUTION

Exercice A

Variable N en Entier

Debut

$N \leftarrow 0$

Ecrire "Entrez un nombre entre 1 et 10"

TantQue $N < 1$ ou $N > 10$

Lire N

Si $N < 1$ ou $N > 10$ **Alors**

Ecrire "Saisie erronée. Recommencez"

FinSi

FinTantQue **Fin**

Exercice B

Variable N en Entier

Debut

$N \leftarrow 0$

Ecrire "Entrez un nombre entre 10 et 50"

TantQue $N < 10$ ou $N > 50$

Lire N

Si $N < 10$ **Alors**

Ecrire "Plus grand !"

SinonSi $N > 50$ **Alors**

Ecrire "Plus petit !"

FinSi

FinTantQue

Fin

Exercice C

On peut imaginer deux variantes, strictement équivalentes :

Variables N, i en Entier

Debut

Ecrire "Saisir le nombre : "

Lire N

$Plage \leftarrow N+10$

ALGORITHMIQUE :EXERCICES SUR LES BOUCLES
TRAVAIL DE GROUPE
EXECUTEZ CHAQUE EXERCICE EN ALGOBOX, EN PYTHON, EN C OU EN JAVA ET
EXPLIQUEZ CHACUNE DES INSTRUCTIONS Ecrire

"Les 10 nombres suivants sont : " **TantQue** N <

Plage

N ← N+1

Ecrire N

FinTantQue

Fin

Ou bien :

Variables N, i **en Entier**

Debut

Ecrire "Saisir le nombre : "

Lire N i

← 0

Ecrire "Les 10 nombres suivants sont : "

TantQue i < 10

i ← i + 1

Ecrire N + i

FinTantQue

Fin

Exercice D

Là encore, deux variantes, correspondant trait pour trait à celles du corrigé précédent :

Variables N, i **en Entier**

Debut

Ecrire "Entrez un nombre : "

Lire N

Ecrire "Les 10 nombres suivants sont : "

Pour i ← N + 1 à N + 10

Ecrire i

i Suivant

Fin

Ou bien :

Variables N, i **en Entier**

Debut

Ecrire "Entrez un nombre : "

Lire N

"Les 10 nombres suivants sont : "

ALGORITHMIQUE :EXERCICES SUR LES BOUCLES
TRAVAIL DE GROUPE
EXECUTEZ CHAQUE EXERCICE EN ALGOBOX, EN PYTHON, EN C OU EN JAVA ET
EXPLIQUEZ CHACUNE DES INSTRUCTIONS Ecrire

Pour $i \leftarrow 1$ à 10

Ecrire $N + i$

i Suivant Fin

Exercice E

Variables N, i **en Entier**

Debut

Ecrire "Entrez un nombre : "

Lire N

Ecrire "La table de multiplication de ce nombre est : "

Pour $i \leftarrow 1$ à 10

Ecrire N, " x ", i, " = ", $n*i$

i Suivant Fin

Exercice F

Variables N, i, Som **en Entier**

Debut

Ecrire "Entrez un nombre : "

Lire N

Som $\leftarrow 0$

Pour $i \leftarrow 1$ à N

 Som $\leftarrow$ Som + i

Suivant

Ecrire "La somme est : ", Som

Fin

Exercice G

Variables N, i, F **en Entier**

Debut

Ecrire "Entrez un nombre : "

Lire N

F $\leftarrow 1$

Pour $i \leftarrow 2$ à N

F $\leftarrow F * i$

Suivant

 "La factorielle est : ", F

Fin

ALGORITHMIQUE :EXERCICES SUR LES BOUCLES
TRAVAIL DE GROUPE
EXECUTEZ CHAQUE EXERCICE EN ALGOBOX, EN PYTHON, EN C OU EN JAVA ET
EXPLIQUEZ CHACUNE DES INSTRUCTIONS Ecrire

Exercice H

Variables N, i, PG en Entier

Debut

PG $\leftarrow$ 0

Pour i $\leftarrow$ 1 à 50

Ecrire "Entrez un nombre : "

Lire N

Si i = 1 ou N > PG **Alors**

 PG $\leftarrow$ N

FinSi i

Suivant

Ecrire "Le nombre le plus grand était : ", PG

Fin

En ligne 3, on peut mettre n'importe quoi dans PG, il suffit que cette variable soit affectée pour que le premier passage en ligne 7 ne provoque pas d'erreur.

Pour la version améliorée, cela donne :

Variables N, i, PG, IPG en Entier

Debut

PG $\leftarrow$ 0

Pour i $\leftarrow$ 1 à 20

Ecrire "Entrez un nombre : "

Lire N

Si i = 1 ou N > PG **Alors**

 PG $\leftarrow$ N

 IPG $\leftarrow$ i

FinSi i

Suivant

Ecrire "Le nombre le plus grand était : ", PG

Ecrire "Il a été saisi en position numéro ", IPG **Fin**

Exercice I

Variables N, i, PG, IPG **en Entier**

Debut

N ←

i ← 1

PG ← 0

IPG ← 0

TantQue N <> 0

Ecrire "Entrez un nombre : "

Lire

 N ← i

 i ← i + 1

Si i = 1 **ou** N > PG **Alors**

 PG ← N

 IPG ← i

FinSi

FinTantQue

Ecrire "Le nombre le plus grand était : ", PG

Ecrire "Il a été saisi en position numéro ", IPG **Fin**

Exercice J

Variables E, somdue, M, Reste, Nb10F, Nb5F **En Entier**

Debut F ← 1

somdue ← 0

TantQue E <>

0

Ecrire "Entrez le montant : "

Lire F

 somdue ← somdue + F

FinTantQue

Ecrire "Vous devez :", somdue, " Francs"

Ecrire "Montant versé :"

Lire M

Reste ← M - somdue

Nb10F ← 0

TantQue Reste >= 10

 F ← Nb10F + 1

 Reste ← Reste - 10

FinTantQue

Nb5T ← 0

Si Reste $\geq$ 5
 Nb5F $\leftarrow$ 1
 Reste $\leftarrow$ Reste – 5
FinSi
Ecrire "Rendu de la monnaie :"
Ecrire "Billets de 10000 F : ", Nb10F
Ecrire "Billets de 5000 F : ", Nb5F
Ecrire "Pièces de 1000 F : ", reste
Fin

Exercice K

Spontanément, on est tenté d'écrire l'algorithme suivant :

Variables N, P, i, Numé, Déno1, Déno2 en Entier
 Debut **Ecrire** "Entrez le nombre de chevaux partants : "
Lire N
Ecrire "Entrez le nombre de chevaux joués : "
Lire P
 Numé $\leftarrow$ 1
Pour i $\leftarrow$ 2 à N
 Numé $\leftarrow$ Numé *
 i i **Suivant**
 Déno1 $\leftarrow$ 1
Pour i $\leftarrow$ 2 à N-P
 Déno1 $\leftarrow$ Déno1 *
 i i **Suivant**
 Déno2 $\leftarrow$ 1
Pour i $\leftarrow$ 2 à P
 Déno2 $\leftarrow$ Déno2 *
 i
 i **Suivant**
Ecrire "Dans l'ordre, une chance sur ", Numé / Déno1
Ecrire "Dans le désordre, une sur ", Numé / (Déno1 * Déno2)
Fin

Variables N, P, i, A, B en Numérique

Debut
Ecrire "Entrez le nombre de chevaux partants : "
Lire N
Ecrire "Entrez le nombre de chevaux joués : "
Lire P
 A $\leftarrow$ 1
 B $\leftarrow$ 1

Pour $i \leftarrow 1$ à P

$A \leftarrow A * (i + N -$

$P) \quad B \leftarrow B * i$

Suivant

Ecrire "Dans l'ordre, une chance sur ", A

Ecrire "Dans le désordre, une chance sur ", A / B

Fin

LES TABLEAUX

Exercice T1

Ecrire un algorithme qui déclare et remplit un tableau de 10 valeurs numériques en affectant la note 5 partout.

Exercice T2

Ecrire un algorithme qui déclare et remplit un tableau contenant les six voyelles de l'alphabet latin.

Exercice T3

Ecrire un algorithme qui déclare un tableau de 8 notes, dont on fait ensuite saisir les valeurs par l'utilisateur.

Exercice T4

Que produit l'algorithme suivant ?

Tableau bloc[5] en Entier

Variable i en Entier

Début

Pour i $\leftarrow$ 0 à 5

 bloc [i] $\leftarrow$ i * i

i suivant

Pour i $\leftarrow$ 0 à 5

Ecrire bloc[i]

i suivant

Fin

Peut-on simplifier cet algorithme avec le même résultat ?

Exercice T5

Que produit l'algorithme suivant ?

Tableau N[6] en Entier

Variables i, k en Entier

Début

N[0] $\leftarrow$ 1

Pour k $\leftarrow$ 1 à 6

 N[k] $\leftarrow$ N[k-1] + 2

k Suivant

Pour i $\leftarrow$ 0 à 6

Ecrire N[i]

i suivant

Fin

Peut-on simplifier cet algorithme avec le même résultat ?

Exercice T6

Que produit l'algorithme suivant ?

Tableau Suite[7] en Entier

Variable i en Entier

Début

Suite[0] $\leftarrow$ 1

Suite[1] $\leftarrow$ 1

Pour i $\leftarrow$ 2 à 7

Suite[i] $\leftarrow$ Suite[i-1] + Suite[i-2]

i suivant

Pour i $\leftarrow$ 0 à 7

Ecrire Suite[i]

i suivant

Fin

Exercice T7

Ecrivez la fin de l'algorithme T3 afin que le calcul de la moyenne des notes soit effectué et affiché à l'écran.

Exercice T8

Ecrivez un algorithme permettant à l'utilisateur de saisir un nombre quelconque de valeurs, qui devront être stockées dans un tableau. L'utilisateur doit donc commencer par entrer le nombre de valeurs qu'il compte saisir. Il effectuera ensuite cette saisie. Enfin, une fois la saisie terminée, le programme affichera le nombre de valeurs négatives et le nombre de valeurs positives.

Exercice T9

Ecrivez un algorithme calculant la somme des valeurs d'un tableau (on suppose que le tableau a été préalablement saisi).

Exercice T10

Ecrivez un algorithme constituant un tableau, à partir de deux tableaux de même longueur préalablement saisis. Le nouveau tableau sera la somme des éléments des deux tableaux de départ.

Tableau 1 :

8	11	4	3	17	11	13	2
---	----	---	---	----	----	----	---

Tableau 2 :

1	2	22	7	1	6	20	4
---	---	----	---	---	---	----	---

Tableau à concevoir :

9	11	26	10	18	17	33	6
---	----	----	----	----	----	----	---

Exercice T11

Toujours à partir de deux tableaux précédemment saisis, écrivez un algorithme qui calcule le schtroumpf des deux tableaux. Pour calculer le schtroumpf, il faut multiplier chaque élément du tableau 1 par chaque élément du tableau 2, et additionner le tout. Par exemple si l'on a :

Tableau 1 :

4	8	7	12
---	---	---	----

Tableau 2 :

3	6
---	---

Le Schtroumpf sera :

$$3 * 4 + 3 * 8 + 3 * 7 + 3 * 12 + 6 * 4 + 6 * 8 + 6 * 7 + 6 * 12 = 279$$

Exercice T12

Ecrivez un algorithme qui permette la saisie d'un nombre quelconque de valeurs, sur le principe de l'ex T8. Toutes les valeurs doivent être ensuite augmentées de 1, et le nouveau tableau sera affiché à l'écran.

Exercice T13

Ecrivez un algorithme permettant, toujours sur le même principe, à l'utilisateur de saisir un nombre déterminé de valeurs. Le programme, une fois la saisie terminée, renvoie la plus grande valeur en précisant quelle position elle occupe dans le tableau. On prendra soin d'effectuer la saisie dans un premier temps, et la recherche de la plus grande valeur du tableau dans un second temps.

Exercice T14

Toujours et encore sur le même principe, écrivez un algorithme permettant, à l'utilisateur de saisir les notes d'une promotion. Le programme, une fois la saisie terminée, renvoie le nombre de ces notes supérieures à la moyenne de la promotion.

RESOLUTION

Exercice T1

Tableau L2 [10] en Numérique

Variable i en Numérique

Début

Pour i $\leftarrow$ 0 à 10

 L2 [i] $\leftarrow$ 5

i Suivant

Fin

Exercice T2

Tableau L2 [5] en Caractère

Debut

L2[0] $\leftarrow$ "a"

L2[1] $\leftarrow$ "e"

L2[2] $\leftarrow$ "i"

L2[3] $\leftarrow$ "o"

L2[4] $\leftarrow$ "u"

L2[5] $\leftarrow$ "y"

Fin

Exercice T3

Tableau Notes[7] en Numérique

Variable i en Numérique

Pour i $\leftarrow$ 0 à 7

Ecrire "Entrez la note numéro ", i + 1

Lire Notes[i]

i Suivant

Fin

Exercice T4

Cet algorithme remplit un tableau avec six valeurs : 0, 1, 4, 9, 16, 25.

Il les écrit ensuite à l'écran. Simplification :

Tableau Nb[5] en Numérique

Variable i en Numérique

Début

Pour i $\leftarrow$ 0 à 5

 Nb[i] $\leftarrow$ i * i

Ecrire Nb[i]

i Suivant

Fin

Exercice T5

Cet algorithme remplit un tableau avec les sept valeurs : 1, 3, 5, 7, 9, 11, 13.

Il les écrit ensuite à l'écran. Simplification :

Tableau N[6] en Numérique

Variables i, k en Numérique

Début

```

N[0] ← 1
Ecrire N[0]
Pour k ← 1 à 6
    N[k] ← N[k-1] + 2
    Ecrire N[k]
k Suivant
Fin

```

Exercice T6

Cet algorithme remplit un tableau de 8 valeurs : 1, 1, 2, 3, 5, 8, 13, 21

Exercice T7

```

Variable S en Numérique
Tableau Notes[8] en Numérique
Variable i en Numérique
Debut
s ← 0
Pour i ← 0 à 8
    Ecrire "Entrez la note n° ", i + 1
    Lire Notes[i]
    s ← s + Notes[i]
i Suivant
Ecrire "Moyenne :", s/9
Fin

```

Exercice T8

```

Variables Nb, Nbpos, Nbneg en Numérique
Tableau T[] en Numérique
Variable i en Numérique
Debut
Ecrire "Entrez le nombre de valeurs :"
Lire Nb
Redim T[Nb-1]
Nbpos ← 0
Nbneg ← 0
Pour i ← 0 à Nb - 1
    Ecrire "Entrez le nombre n° ", i + 1
    Lire T[i]
    Si T[i] > 0 alors
        Nbpos ← Nbpos + 1
    Sinon
        Nbneg ← Nbneg + 1
    Finsi
i Suivant
Ecrire "Nombre de valeurs positives : ", Nbpos
Ecrire "Nombre de valeurs négatives : ", Nbneg
Fin

```

Exercice T9

Variables i, Som, N **en Numérique**

Tableau T[] **en Numérique**

Debut

... (on ne programme pas la saisie du tableau, dont on suppose qu'il compte N éléments]

Redim T[N-1]

...

Som $\leftarrow$ 0

Pour i $\leftarrow$ 0 à N - 1

Som $\leftarrow$ Som + T[i]

i Suivant

Ecrire "Somme des éléments du tableau : ", Som

Fin

Exercice T10

Variables i, N **en Numérique**

Tableaux T1[], T2[], T3[] **en Numérique**

Debut

... (on suppose que T1 et T2 comptent N éléments, et qu'ils sont déjà saisis)

Redim T3[N-1]

...

Pour i $\leftarrow$ 0 à N - 1

T3[i] $\leftarrow$ T1[i] + T2[i]

i Suivant

Fin

Exercice T11

Variables i, j, N1, N2, S **en Numérique**

Tableaux T1[], T2[] **en Numérique**

Debut

... On ne programme pas la saisie des tableaux T1 et T2.

On suppose que T1 possède N1 éléments, et que T2 en possède T2)

...

S $\leftarrow$ 0

Pour i $\leftarrow$ 0 à N1 - 1

Pour j $\leftarrow$ 0 à N2 - 1

S $\leftarrow$ S + T1[i] * T2[j]

j Suivant

i Suivant

Ecrire "Le schtroumpf est : ", S

Fin

Exercice T12

Variables Nb, i en Numérique
Tableau T[] en Numérique
Debut
Ecrire "Entrez le nombre de valeurs : "
Lire Nb
Redim T[Nb-1]
Pour i $\leftarrow$ 0 à Nb - 1
 Ecrire "Entrez le nombre n° ", i + 1
 Lire T[i]
i Suivant
Ecrire "Nouveau tableau : "
Pour i $\leftarrow$ 0 à Nb - 1
 T[i] $\leftarrow$ T[i] + 1
 Ecrire T[i]
i Suivant
Fin

Exercice T13

Variables i, Nb, Posmaxi en Numérique
Tableau T[] en Numérique
Ecrire "Entrez le nombre de valeurs :"
Lire Nb
Redim T[Nb-1]
Pour i $\leftarrow$ 0 à Nb - 1
 Ecrire "Entrez le nombre n° ", i + 1
 Lire T[i]
i Suivant
 Posmaxi $\leftarrow$ 0
Pour i $\leftarrow$ 0 à Nb - 1
 Si T[i] > T[Posmaxi] **alors**
 Posmaxi $\leftarrow$ i
 Finsi
i Suivant
Ecrire "Element le plus grand : ", T[Posmaxi]
Ecrire "Position de cet élément : ", Posmaxi
Fin

Exercice T14

Variables Nb, i, Som, Moy, Nbsup en Numérique
Tableau T[] en Numérique
Debut
Ecrire "Entrez le nombre de notes à saisir : "
Lire Nb
Redim T[Nb-1]
Pour i $\leftarrow$ 0 à Nb - 1
 Ecrire "Entrez le nombre n° ", i + 1
 Lire T[i]

```
i Suivant  
Som  $\leftarrow$  0  
Pour i  $\leftarrow$  0 à Nb - 1  
    Som  $\leftarrow$  Som + T[i]  
i Suivant  
Moy  $\leftarrow$  Som / Nb  
NbSup  $\leftarrow$  0  
Pour i  $\leftarrow$  0 à Nb - 1  
    Si T[i] > Moy Alors  
        NbSup  $\leftarrow$  NbSup + 1  
    FinSi  
i Suivant  
Ecrire NbSup, " étudiants dépassent la moyenne de la promotion"  
Fin
```

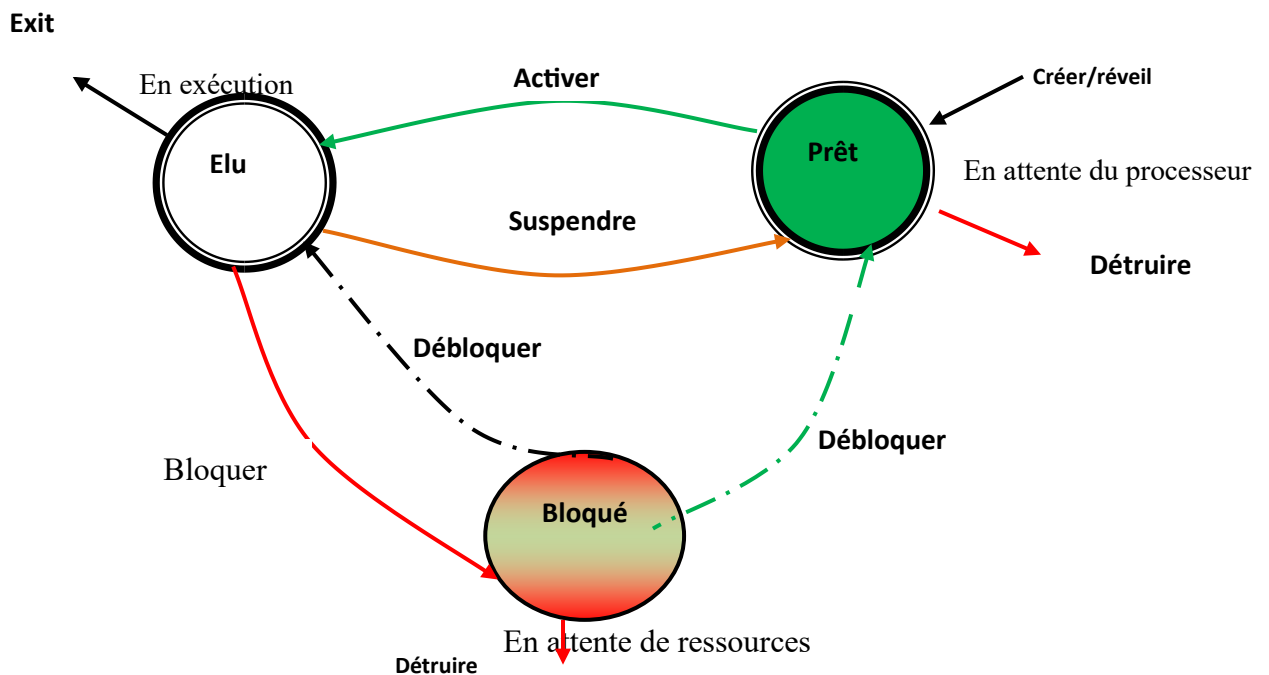
Les algorithmes des Systèmes d'exploitation

■ Diagramme de transition d'état d'un processus

Un processus est donc une entité dynamique qui a une durée de vie limitée (la durée de l'exécution du programme) et dont on peut caractériser le comportement à un certain niveau d'abstraction par un diagramme de transitions d'état.

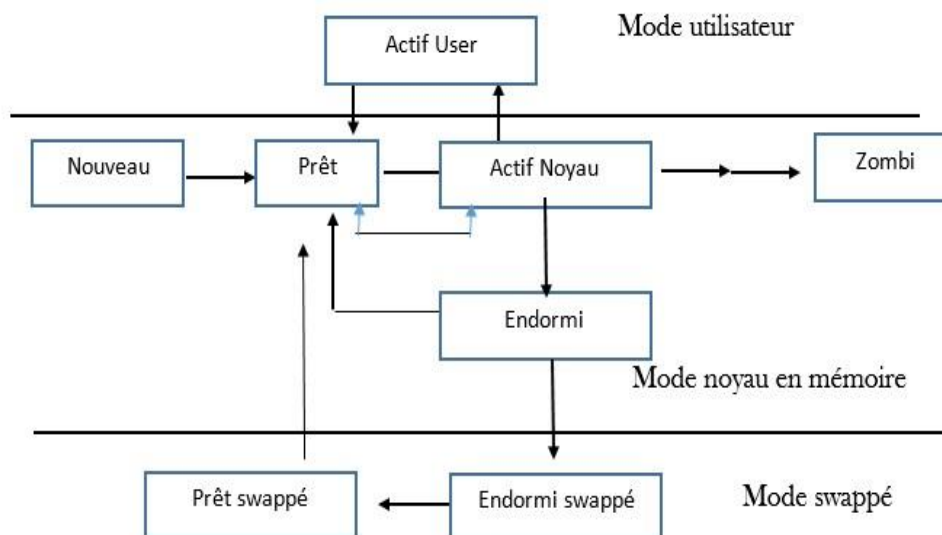
- Dans l'état actif, le processus exécute des instructions d'un programme. Il monopolise un processeur.
- Dans l'état prêt, le processus est arrêté. Il ne lui manque que la ressource processeur pour devenir actif. Autrement dit, il suffira de lui allouer le (un) processeur et de charger le contexte d'exécution sauvegardé pour que celui-ci poursuive l'exécution du programme dont il a la charge.
- Dans l'état bloqué, le processus est aussi arrêté. Mais, pour se poursuivre, une condition logique devra de plus devenir vraie. Par exemple, le processus interpréteur attendant qu'une ligne de commande soit frappée est bloqué tant que cette ligne n'est pas disponible. La sortie de l'état bloqué pourra conduire le processus soit dans l'état actif, soit dans l'état prêt. Ce choix est fixé par la stratégie d'ordonnancement des processus vis-à-vis de la ressource processeur.

On désigne souvent l'ensemble composé des processus prêts et actifs comme l'ensemble des processus exécutables.



Un processus Unix évolue entre trois modes au cours exécution : le mode utilisateur qui est le mode classique d'exécution, le mode noyau en mémoire qui est le mode dans lequel se trouve un processus élu passé en mode superviseur, un processus prêt ou un processus bloqué(endormi) et le mode swappé qui est le mode dans lequel se trouve un processus bloqué(endormi) déchargé de la mémoire centrale. En effet, le système Unix décharge de la mémoire centrale les processus endormis depuis trop longtemps (ils sont alors dans l'état endormi swappé). Ces processus réintègrent la mémoire centrale lorsqu'ils redeviennent prêts (transition de l'état prêt swappé vers l'état prêt).

Un processus Unix qui se termine passe dans un état zombi.



■ Les opérations sur les processus

Le diagramme de transitions fait apparaître les primitives applicables aux objets processus. Celles-ci définissent la classe ou le type des objets processus du point de vue du noyau de gestion des processus.

▪ Créer

La première opération indispensable est l'opération de création d'un processus. Pour le noyau, un objet processus est un objet comme un autre. La création d'un processus apparaît donc comme la création d'un objet de la classe processus avec ses attributs et méthodes (opérations). Parmi les attributs d'un processus, on trouve par exemple, le nom du fichier d'où est issu le programme associé au processus, un nom interne de processus (numéro par exemple), l'état, le contexte initial qui sera chargé dans les registres du processeur pour débiter l'exécution, l'espace mémoire alloué pour le chargement et l'exécution du programme, certains paramètres d'exécution (priorité, délais de garde, . . .), le nom de l'utilisateur pour lequel le processus « travaille », l'environnement fichier accessible (répertoire de travail), etc. Lorsqu'on crée un processus, deux approches sont possibles : soit le nouvel objet est créé de toute pièce, soit le nouvel objet est une copie d'un processus courant. Cette dernière approche est par exemple adoptée par le système Unix. Elle présente l'avantage de pouvoir décomposer la création d'un processus en deux étapes :

- Une première étape concerne l'aspect parallèle avec la création d'un processus sosie et
- Une deuxième étape concerne l'aspect traitement (programme) avec la commutation du programme exécuté par le processus sosie.

□ Activer

Un processus prêt peut devenir actif par l'opération Activer. Cette opération consiste essentiellement, à charger dans les registres du processeur le contexte sauvegardé en mémoire. La ressource processeur est ainsi allouée au processus pour exécuter réellement un programme. La période active du processus se termine soit par un blocage, soit par une préemption du processeur au bout d'un délai fixé maximal.

□ Suspendre

Si la commutation d'un processus à un autre n'est causée que pas le blocage d'un processus actif, un processeur peut être monopolisé par un processus pendant une longue période. Par exemple, si le programme exécuté comporte de longues étapes de calcul sur des données en mémoire centrale, il faudra attendre une lecture ou écriture sur disque pour qu'une commutation se produise. Pour mieux répartir le temps processeur entre les processus, un processeur est en général alloué au processus pour une durée maximale fixée appelée quantum. Si le processus atteint la fin de quantum, alors une interruption est provoquée et le processus actif sur le processeur interrompu est suspendu. Libérant ainsi le processeur, un autre processus prêt peut être activé.

□ Bloquer

Cette opération comporte une première étape identique à l'opération Suspendre. Mais, le processus ne peut plus être candidat à la ressource processeur. Il devra être replacé dans l'état prêt, éventuellement directement dans l'état actif, par une opération explicite Débloquer. Les périodes pendant lesquelles un processus est bloqué correspondent aux périodes pendant lesquelles la poursuite de l'activité du processus est conditionnée par la disponibilité d'une ressource ou l'occurrence d'un événement (fin d'entrée/sortie par exemple).

Cette opération a un effet de bord important : elle libère la ressource processeur pour un autre processus prêt. Par conséquent, elle conduit le noyau à enchaîner par une opération Activer (si possible).

□ Débloquer

Cette opération replace un processus parmi les processus exécutables. Elle est invoquée en général sur occurrence d'un événement asynchrone provoqué par un autre processus, une fin d'entrée/sortie ou une fin de quantum.

□ Exit

La terminaison d'un programme se traduit par la fin du processus mais pas l'arrêt du processeur. Par conséquent, la fin du programme doit se traduire par un appel explicite au noyau d'exécution via une primitive Exit. L'objet processus correspondant pourra alors

être détruit sauf contraintes particulières. Une commutation de processus s'ensuivra automatiquement.

□ Détruire

Si un programme ne se termine pas normalement ou si une raison externe nécessite de détruire un processus (manque de ressource, surcharge du système), on pourra être amené à détruire un processus non actif.

▪ Ordonnancement des tâches

Définition

L'ordonnancement c'est un Mécanisme par lequel on donne la main successivement à tous les processus *prêts*, dans un système multitâche. C'est l'ordonnanceur qui met en œuvre l'ordonnancement.

Quelques algorithmes utilisés

Dans cette partie, nous allons voir quelques algorithmes d'ordonnancement simples. Bien sûr, ces algorithmes sont rarement utilisés tels quels dans les systèmes d'exploitations modernes, qui utilisent des algorithmes d'ordonnements bien plus complexes et souvent plus efficaces.

a) Algorithmes non collaboratifs

Tout d'abord nous allons commencer par un système d'exploitation qui exécute des programmes qui s'exécutent les uns après les autres. Ces programmes sont exécutés jusqu'à ce qu'ils finissent leur travail ou décident eux-mêmes de stopper leur exécution, soit pour accéder à un périphérique, soit pour laisser la place à un autre programme.

➡ First Input First Output :FIFO

Avec cet algorithme, les programmes à exécuter sont ajoutés dans la file d'attente quand on les démarre. Ceux-ci sont alors stockés dans la file d'attente dans l'ordre dans lesquels on les a fait démarrer. L'ordonnanceur décide alors d'exécuter le programme entré dans la file d'attente avant tous les autres en premier. En clair, les programmes sont exécutés dans l'ordre dans lequel ils sont rentrés dans la file d'attente.

➡ Last Input First Output :LIFO

Avec cet algorithme, les programmes à exécuter sont ajoutés dans la file d'attente quand on les démarre. Ceux-ci sont alors stockés dans la file d'attente dans l'ordre dans lesquels on les a fait démarrer. Mais contrairement à l'algorithme First Input First Output, l'ordonnanceur exécute le programme entré en dernier dans la file d'attente, et non le premier. Si nous ajoutons un programme dans une file d'attente contenant déjà des programmes en attente, ce sera lui le prochain à être exécuté, vu qu'il a été ajouté en dernier. Cet algorithme peut souffrir d'un phénomène assez particulier : dans certains cas, un programme peut très bien mettre énormément de temps avant d'être exécuté.

➤ Shortest Job First

Cet algorithme est basé sur une logique simple. Les programmes à exécuter sont placés dans la file d'attente et l'ordonnanceur va alors décider d'exécuter ces processus dans l'ordre. Pour appliquer cet algorithme, on suppose que les temps d'exécution des différents programmes sont connus à l'avance et sont parfaitement bornés. Cette contrainte peut sembler absurde, mais elle a un sens dans certains cas assez rares dans lesquels on connaît à l'avance le temps mis par un programme pour s'exécuter. Dans ce cas, l'algorithme est simple : on exécute le programme qui met le moins de temps à s'exécuter en premier. Une fois celui-ci terminé, on le retire du fil d'attente et on recommence.

b) Algorithmes préemptifs

➤ Shortest Remaining Time Next:

C'est une variante préemptive de l'algorithme Shortest Job First vu audessus. Dans cette version, si un programme est ajouté dans la file d'attente, on regarde le temps que ce nouveau venu mettrait à s'exécuter, et on compare avec le temps qu'il reste au processus en cours d'exécution avant que celui-ci finisse son travail. Si le temps mis par le nouveau programme est plus faible que le temps d'exécution du programme en train de s'exécuter, on change et on exécute le nouveau venu à la place.

➤ Fixed priority pre-emptive scheduling

Avec cet algorithme, chaque programme possède une priorité : certains programmes seront donc prioritaires sur les autres. Cette priorité est définie par un nombre. Cette priorité est toujours la même et ne peut pas changer. L'ordonnanceur exécute toujours le programme le plus prioritaire.

➤ Round-robin : Tourniquet

Cet algorithme est le premier de nos algorithmes qui utilise la méthode du *quantum* de temps : on donne à chaque programme un temps fixé une bonne fois pour toute durant lequel il peut s'exécuter. Une fois ce temps fini, l'ordonnanceur stoppe celui-ci et demande l'exécution d'un autre programme. Il se base toujours sur une file d'attente de programmes. Cette liste est ordonnée (comme pour l'algorithme FIFO et LIFO) et a un début et une fin. Tout programme qui vient d'être lancé est placé à la fin de cette liste.

Chaque programme dans la file d'attente a droit à un quantum de temps. Quand un programme a fini son exécution, il doit attendre que tous les autres programmes aient eu droit à leur quantum de temps avant de pouvoir être exécuté une nouvelle fois. Pour cela, à la fin de chaque quantum de temps, on place le programme à la fin de la liste d'attente.

➤ Multilevel feedback queue

Cet algorithme est déjà plus proche de ceux implémentés dans les systèmes d'exploitation actuels. La philosophie derrière cet algorithme est :

- D'ordonnancer aux mieux un ensemble de programmes aux temps d'exécutions et aux particularités disparates ;
- D'exécuter en priorité les programmes rapides ;
- D'exécuter en priorité les programmes qui accèdent souvent aux périphériques.

Le principe de cet algorithme est simple : on dispose de plusieurs files d'attente comme celle de l'algorithme round-robin. Ces files d'attente sont classées de la plus basse à la plus haute. Ces files d'attente ont des priorités différentes. De plus, le quantum de temps est différent suivant la file d'attente : la file d'attente la plus haute a un quantum de temps très petit, tandis que la file la plus basse a un quantum de temps élevé.

■ Ordonnancement des processus

Dans un système multi -utilisateurs à temps partagé, plusieurs processus peuvent être présents en mémoire centrale en attente d'exécution.

Si plusieurs processus sont prêts, le système d'exploitation doit gérer l'allocation du processeur aux différents processus à exécuter. C'est l'ordonnanceur qui s'acquitte de cette tâche.

Tout le logiciel d'un ordinateur peut être vu comme un ensemble de processus dont l'exécution est gérée par un processus particulier : l'ordonnanceur (scheduler en anglais).

Un ordonnanceur fait face à deux problèmes principaux :

- Le choix du processus à exécuter, et
- Le temps d'allocation du processeur au processus choisi.

Un système d'exploitation multitâche est préemptif lorsque celui-ci peut arrêter (réquisition) à tout moment n'importe quelle application pour passer la main à la suivante. Dans les systèmes d'exploitation préemptifs on peut lancer plusieurs applications à la fois et passer de l'une à l'autre, il arrive aussi de lancer une application pendant qu'une autre effectue un travail.

Il y a aussi des systèmes d'exploitation dits multitâches, qui sont en fait des « *multitâches coopératifs* ». Quelle est la différence ? Un multi tâche coopératif permet à plusieurs applications de fonctionner et d'occuper des plages mémoire, laissant le soin à ces applications de gérer cette occupation, au risque de bloquer tout le système. Par contre, avec un « *multitâche préemptif* », le noyau garde toujours le contrôle (qui fait quoi, quand et comment), et se réserve le droit de fermer les applications qui monopolisent les ressources du système. Ainsi les blocages du système sont inexistantes.

I.3.9.4. Objectifs de l'ordonnanceur d'un système multiutilisateur

Les objectifs d'un ordonnanceur d'un système multi utilisateurs ont, entre autres :

- S'assurer que chaque processus en attente d'exécution reçoive sa part de temps processeur ;
- Minimiser le temps de réponse ;
- Utiliser le processeur à 100% ;
- Utilisation équilibrée des ressources ;
- Prendre en compte des priorités ; - Être prédictibles.

Ordonnancement non préemptif

Dans un système à ordonnancement non préemptif ou sans réquisition, le système d'exploitation choisit le prochain processus à exécuter, en général, le Premier Arrivé est le Premier Servi PAPS (ou First-Come First-Served FCFS) ou le plus court d'abord (Short Job First SJF). Il lui alloue le Processeur jusqu'à ce qu'il se termine ou qu'il se bloque (en attente d'un événement). Il n'y a pas de réquisition.

Si l'ordonnanceur fonctionne selon la stratégie SJF, il choisit, parmi le lot de processus à exécuter, le plus court (plus petit temps d'exécution). Cette stratégie est bien adaptée au traitement par lots de processus dont les temps maximaux d'exécution sont connus par les utilisateurs car elle offre un meilleur temps moyen de séjour. Le temps de séjour d'un processus (temps de rotation ou de virement) est l'intervalle de temps entre la soumission du processus et son achèvement.

Toutefois, l'ordonnancement du plus court d'abord est optimal que si les travaux sont disponibles simultanément.

Exemple 1. Considérons cinq travaux Téléchargement, Rendering, compilation, Gravure et Formatage, dont les temps d'exécution et leurs arrivages respectifs sont donnés dans la table ci-dessous. Faire un schéma qui illustre son exécution et calculer le temps de séjour de chaque Processus, le temps moyen de séjour, le temps d'attente et le temps moyen d'attente en utilisant :

1. L'algorithme de Premier arrivé premier servi (FIFO)
2. l'algorithme du plus court d'abord (SJF)

Schéma d'exécution

A	A	A	B	B	B	B	B	B	C	C	C	C	D	D	E
1			2						3				4		5

Processus	Temps d'exécution	Temps d'arrivage
TA	3	0
RB	6	1
C	4	4
GD	2	6
FE	1	7

Au temps 0, seulement le processus A est dans le système et il s'exécute. Au temps 1 le processus B arrive mais il doit attendre que A termine car il a encore 2 unités de temps. Ensuite C s'exécute pendant 4 unités de temps. Au temps 4, 6, et 7 les processus C, D et E arrivent mais C a encore 2 unités de temps. Une fois que C a terminé, D et E entrent au système dans l'ordre 1.

Le temps de séjour pour chaque processus est obtenu soustrayant le temps d'entrée du processus du temps de terminaison. Ainsi :

Processus	Temps de sejour
A	$3-0 = 3$
B	$9-1 = 8$
C	$13-4 = 9$
D	$15-6 = 9$
E	$16-7 = 9$

Le temps moyen de séjour est : $\frac{(3+8+9+9+9)}{5} = \frac{38}{5} = 7,6$

Le temps d'attente est calculé soustrayant le temps d'exécution du temps de séjour :

Processus	Temps d'attente
A	$3-3 = 0$
B	$8-6 = 2$
C	$9-4 = 5$
D	$9-2 = 7$
E	$9-1 = 8$

Le temps moyen d'attente est : $\frac{(0+2+5+7+8)}{5} = \frac{23}{5} = 4,4$

Il y a cinq tâches exécutées dans 16 unités de temps, alors $16/5 = 3,2$ unités de temps par processus.

Processus	Temps de séjour
A	$3-0 = 3$
B	$9-1 = 8$
E	$10-7 = 3$
D	$12-6 = 6$
C	$16-4 = 12$

$$\frac{(3+8+3+6+12)}{5} = \frac{32}{5} = 6,4$$

Processus	Temps d'attente
A	$3-3 = 0$
B	$8-6 = 2$
E	$3-1 = 2$
D	$6-2 = 4$
C	$12-4 = 8$

Le temps moyen d'attente est : $\frac{(0+2+2+4+8)}{5} = \frac{16}{5} = 3,2$

Il y a cinq tâches exécutées dans 16 unités de temps, alors $16/5 = 3,2$ unités de temps par processus.

Exercices sur les opérations mathématiques et la notion des priorités

Exercice 1:

Calculez les expressions suivantes en respectant l'ordre des opérations :

- a) $2 + 3 * 4$
- b) $(2 + 3) * 4$
- c) $10 - 5 / 2$
- d) $(10 - 5) / 2$

Corrigé:

- a) $2 + 3 * 4 = 2 + 12 = 14$
- b) $(2 + 3) * 4 = 5 * 4 = 20$

c) $10 - 5 / 2 = 10 - 2.5 = 7.5$

d) $(10 - 5) / 2 = 5 / 2 = 2.5$

Exercice 2:

Calculez les expressions suivantes en indiquant les étapes de calcul et en justifiant l'ordre des opérations :

a) $5 + 4 * 3 - 2$

b) $(5 + 4) * (3 - 2)$

c) $10 / 2 + 3 * 4$

d) $10 * (2 + 3) / 4$

Corrigé:

a) $5 + 4 * 3 - 2$

Étape 1: Multiplication : $4 * 3 = 12$

Étape 2: Addition : $5 + 12 = 17$

Étape 3: Soustraction : $17 - 2 = 15$

Justification: La multiplication et la division sont effectuées avant l'addition et la soustraction.

b) $(5 + 4) * (3 - 2)$

Étape 1: Addition : $5 + 4 = 9$

Étape 2: Soustraction : $3 - 2 = 1$

Étape 3: Multiplication : $9 * 1 = 9$

Justification: Les opérations entre parenthèses sont effectuées en premier. Ensuite, la multiplication et la division sont effectuées avant l'addition et la soustraction.

c) $10 / 2 + 3 * 4$

Étape 1: Division : $10 / 2 = 5$

Étape 2: Multiplication : $3 * 4 = 12$

Étape 3: Addition : $5 + 12 = 17$

Justification: La division est effectuée avant la multiplication. Ensuite, l'addition est effectuée après la multiplication.

d) $10 * (2 + 3) / 4$

Étape 1: Addition : $2 + 3 = 5$

Étape 2: Multiplication : $10 * 5 = 50$

Étape 3: Division : $50 / 4 = 12.5$

Justification: Les opérations entre parenthèses sont effectuées en premier. Ensuite, la multiplication est effectuée avant la division.

Exercice 3:

Réécrivez les expressions suivantes en utilisant des parenthèses pour obtenir le résultat indiqué :

a) $5 + 4 * 3 = 23$

b) $10 / 2 + 3 * 4 = 22$

Corrigé:

a) $5 + (4 * 3) = 23$

b) $(10 / 2) + (3 * 4) = 22$

Problème

Madame MPEMBA, a acheté un ordinateur Mac book pro à 1200\$, elle l'a vendu à 1650\$. Elle le rachète à 1790\$ et revendu à 2125\$.

- a) Quel est son bénéfice ?
- b) Proposez un algorithme qui calcule son bénéfice tout en gérant les inputs (les variables)

Références bibliographiques

- [1] Andrew TANENBAUM, système d'exploitation ,3^{ème}édition Pearson, paris 2008 ;
- [2] D. BOUCHIHA « Initiation à l'Algorithmique et à la Programmation en Pascal » Errachad, 2019.
- [3] F. DAOUDI « Introduction à l'algorithmique » l'Abeille, 2008.
- [4] M. DIVAY « Algorithmes et structures de données génériques : Cours et exercices corrigés en langage C » 2^{ème} éd. Dunod, 2004.
- [5] S. GRAÏNE « Le langage C avec exercices corrigés » l'Abeille, 2009.
- [6] C. KHICHANE « Le leader de l'Algorithmique - Cours et exercices avec corrections » El Maarifa, 2004.
- [7] D.E. KNUTH « The art of computer programming: Volume 1: Fundamental Algorithms » 3rd Ed. Addison-Wesley, 1997.
- [8] R. MALGOUYRES, R. ZROUR et F. FESCHET « Initiation à l'algorithmique et à la programmation en C : Cours avec 129 exercices corrigés » 2^{ème} éd. Dunod, 2014.
- [9] S. ROHAUT « Algorithmique Techniques fondamentales de programmation » ENI, 2007.
- [10] H. ZEMANEK « Al-khorezmi his background, his personality his work and his influence ». In : Algorithms in Modern Mathematics and Computer Science. Springer, Berlin, Heidelberg, 1981. p. 1-81.
- [11] Abraham Silberschatz, Peter Baer Galvin et Greg Gagne, Operating System
- [12] Concepts, 7^{ième} édition, John Wiley and Sons Inc., 2004.
- [13] Abraham Silberschatz, Peter Baer Galvin et Greg Gagne, Operating
- [14] Andrew TANENBAUM, système d'exploitation ,3^{ème}édition Pearson, paris 2004 ;
- [15] Gérard Padiou, precis de système d'exploitation INP Toulouse 5 octobre 2010.
- [16] T. CORMEN, C. LEISERSON, R. RIVEST et C. STEIN « Introduction à l'algorithmique : Cours et exercices corrigés » 2^{ème} éd. Dunod, 2002.
- [17] P. DAMPHOUSSE « Petite introduction à l'algorithmique : à la découverte des mathématiques du pas à pas » Ellipses, 2005.